

ESD113 — Probabilités et statistique avec R

De l'introduction à R au clustering : Keno, Tidyverse et méthodes non supervisées

Auditeur CNAM : Alban VIDELOUP

3 mai 2026

Table des matières

Remerciements	3
1 Introduction générale	5
Partie I — Introduction à R, Quarto et Tidyverse	7
2 Initiation à R, Quarto et Markdown	7
2.1 Présentation générale	7
2.2 Mise en forme du texte en Markdown	7
2.2.1 Formules mathématiques	7
2.3 Les chunks de code R	8
2.4 Références bibliographiques dans Quarto	9
3 Le Tidyverse	9
3.1 Présentation	9
3.2 Manipulation d'un data frame en R de base	9
3.3 Manipulations avec le Tidyverse et le pipe <code> ></code>	10
3.4 Statistiques descriptives avec le Tidyverse	12
4 Introduction à la visualisation avec R	13
4.1 Histogramme de base (R natif)	13
4.2 Histogramme avec ggplot2	14
Partie II — Projet Keno : analyse des données FDJ	16
5 Projet Keno — Analyse des données FDJ	16
5.1 Présentation du Keno	16
5.2 Chargement des données	16
5.3 Pivot longer : restructuration des données	16
5.3.1 Le concept de pivot	16
5.3.2 Application au Keno	17
5.4 Tableau de fréquence des boules	18
5.5 Formatage des dates	18
5.6 Palmarès des numéros — style FDJ	18

5.7	Statistiques sur le format long	22
6	Visualisations graphiques du Keno	24
6.1	Histogramme des fréquences	24
6.2	Graphique circulaire (diagramme en rose)	25
7	Modélisation probabiliste du Keno	27
7.1	La loi hypergéométrique	27
7.2	Calcul des probabilités de gain	27
7.3	Tableau des gains officiels FDJ	28
8	Référence officielle FDJ : tableau complet des gains	28
8.1	Les tableaux de gains officiels	28
8.2	Construction de la table complète des gains FDJ	30
8.3	Espérances de gain par grille	33
9	Espérance de gain	36
9.1	Définition et formule	36
9.2	Calcul avec R	36
10	Simulations de Monte-Carlo	37
10.1	Principe	37
10.2	Premier exemple : simulation simple	37
10.3	Estimation d'une probabilité par Monte-Carlo	37
10.4	Simulation de Monte-Carlo à grande échelle (1 000 itérations)	38
Partie III — Méthodes de clustering		41
11	Introduction au clustering	41
11.1	Qu'est-ce que le clustering ?	41
11.2	Bref panorama historique	41
11.2.1	La petite (et longue) histoire de l'algorithme de Lloyd	41
11.3	Objectifs et plan du volet clustering	42
12	Préparation de l'environnement R pour le clustering	42
13	Simulation des données : créer un laboratoire contrôlé	42
13.1	Pourquoi simuler ?	42
13.2	Fondements mathématiques : la loi normale multivariée	43
13.2.1	La loi normale univariée	43
13.2.2	La loi normale multivariée	43
13.2.3	Le modèle de mélange gaussien	43
13.3	Simulation des variables continues	44
13.3.1	Vers une lecture plus nette : l'enveloppe convexe	44
13.3.2	Reproduction des couleurs originales de Flynt & Dean (2016)	44
14	Méthodes de partitionnement géométrique	46
14.1	K-means : la méthode des barycentres	46
14.1.1	Principe et intuition	46
14.1.2	Formalisation mathématique	47
14.1.3	Choisir K : la méthode du coude	47
14.1.4	Six variations pour générer la courbe WSS	48

14.2	Superposition K-means / vérité terrain	54
14.2.1	Performance du K-means (K=3)	55
14.3	Classification hiérarchique ascendante (CHA)	57
14.3.1	Principe	57
14.3.2	La méthode hiérarchique CAH sur les données simulées	58
14.3.3	La version circulaire	59
14.3.4	Le calcul (30 % d'erreur)	60
15	Méthodes basées sur les modèles probabilistes	61
15.1	Modèles de mélanges gaussiens : Mclust	61
15.1.1	Du déterministe au probabiliste	61
15.1.2	L'algorithme EM	61
15.1.3	Sélection automatique via le BIC	62
16	Comparaison et synthèse des méthodes	67
17	Conclusion générale	68
17.1	Bilan du parcours : de R aux méthodes non supervisées	68
17.2	Ce que les données simulées nous ont appris	69
17.3	La hiérarchie de performance (contexte-dépendante)	69
17.4	Applications concrètes du clustering dans la vie réelle	70
17.5	Pour aller plus loin	70
	Références bibliographiques	71
	Rapport de compilation	71
	Méthode de mesure du temps	71
	Environnement système	71
	Temps d'exécution des chunks R (Phase 2)	72
	Bilan global des phases	74
	Bibliographie	74

Remerciements

 À celles et ceux sans qui rien de tout cela n'aurait été possible

À **Rhizlane**, mon épouse.

Les soirées passées sur ces pages, les week-ends confisqués par les algorithmes et les compilations récalcitrantes — tu les as vécus avec une patience et une générosité qui forcent l'admiration. Tu as tenu la maison, organisé les journées, rassuré les enfants qui demandaient où était papa, et tu l'as fait avec cette force tranquille qui te caractérise. Ce travail t'appartient autant qu'à moi. Merci d'avoir cru, sans jamais fléchir, que ces efforts en valaient la peine.

À mes deux enfants,

Vous avez souvent dû vous passer de moi à des moments où un père devrait être présent — les jeux du soir, les histoires du coucher, les repas en famille que l'écran a parfois remplacés. Vous avez accepté ces absences avec une maturité touchante, et chacun de vos sourires, chacune de vos questions sur « ce que fait papa sur l'ordinateur », m'a redonné l'énergie de continuer. Je vous dois du temps, de la présence, et je m'y engage. Ce document, c'est aussi votre victoire.

À Monsieur Karim KILANI, responsable de l'unité d'enseignement ESD113,

Rares sont les enseignants qui parviennent à rendre la statistique non seulement accessible, mais véritablement désirable. Votre pédagogie rigoureuse, votre disponibilité et votre souci constant de contextualiser les méthodes ont transformé ce cours en une exploration intellectuelle passionnante. Merci de consacrer votre expertise à des auditeurs comme nous, qui jonglent entre vie professionnelle et ambition académique.

Au CNAM,

Permettre à un auditeur qui approche la cinquantaine, au cœur d'une vie active intense, de se former aux méthodes quantitatives avancées, à la data science, à R et à Quarto — c'est un acte de foi dans l'idée que l'éducation n'a pas d'âge et que l'apprentissage tout au long de la vie n'est pas un slogan mais une réalité concrète. Le CNAM incarne cette conviction avec une constance exemplaire depuis plus de deux siècles. Il est difficile d'exprimer combien ce dispositif représente, pour des professionnels comme moi, une chance inestimable de rester dans la course d'un monde qui accélère. Merci d'exister, merci de persévérer, merci d'accueillir ceux que les circuits classiques ne peuvent plus atteindre.

Alban VIDELOUP

Paris, 03 May 2026

1 Introduction générale

Ce document a été réalisé dans le cadre de l'unité d'enseignement **ESD113 — Probabilités et statistiques avec R**, dispensée au Conservatoire National des Arts et Métiers (CNAM) par Monsieur Karim KILANI.

Un recours significatif aux outils d'intelligence artificielle a permis de résoudre les problématiques complexes de mise en page et de structuration du contenu.

Ce document fusionne trois volets complémentaires du cours :

1. **Volet 1 — Introduction à R, Quarto et Tidyverse** : prise en main de l'environnement de travail, manipulation de données, visualisation.
2. **Volet 2 — Projet Keno** : application à un cas concret avec les données de tirage du Keno (FDJ), incluant modélisation probabiliste et simulation de Monte-Carlo.
3. **Volet 3 — Méthodes de clustering** : reproduction et adaptation du code R de l'article de référence « *A Survey of Popular R Packages for Cluster Analysis* » (Flynt & Dean, 2016), couvrant K-means, classification hiérarchique ascendante et modèles de mélanges gaussiens (Mclust).

Le fil conducteur est la **rigueur reproductible** : chaque résultat est produit directement par le code R présenté, dans un document Quarto compilable.

💡 Note sur le format Typst

Ce document est la version **Typst** du rapport. Typst offre une composition typographique de haute qualité, comparable à LaTeX, tout en assurant des temps de compilation nettement plus rapides. Les formules mathématiques sont rendues nativement par le moteur Typst ; les callouts Quarto, les tableaux et les graphiques R sont pleinement pris en charge. Pour chaque formule, le bloc grisé affiche le **code LaTeX source** à titre pédagogique, et le rendu apparaît immédiatement en dessous.

Toutes les analyses sont réalisées avec R [1] et Quarto [2].

Partie I — Introduction à R, Quarto et Tidyverse

2 Initiation à R, Quarto et Markdown

2.1 Présentation générale

Ce premier volet est une introduction pratique au langage **R** et à l'environnement de publication **Quarto**. L'objectif est triple :

1. **Découvrir Quarto** : un système de publication scientifique qui mêle texte, code et résultats dans un seul document.
2. **Maîtriser les bases de R et du Tidyverse** : manipulation de données, statistiques descriptives, visualisation.
3. **Appliquer ces outils à un cas réel** : les données de tirage du **Keno FDJ**, avec modélisation probabiliste et simulation de Monte-Carlo.

i Analogie pédagogique

Quarto, c'est comme un cahier de laboratoire numérique : on y écrit ses observations (le texte), on y insère ses protocoles (le code R), et les résultats apparaissent automatiquement à la compilation. Plus besoin de copier-coller les sorties manuellement.

2.2 Mise en forme du texte en Markdown

Quarto repose sur la syntaxe **Markdown** pour la mise en forme du texte. On peut écrire du texte en **gras** et en *italique*, insérer des listes, des tableaux, et bien sûr des formules mathématiques grâce à LaTeX.

2.2.1 Formules mathématiques

Pour chaque formule, le bloc grisé montre le **code LaTeX source** tel qu'il apparaît dans le document .qmd, et le rendu est affiché juste en dessous. Cette double présentation est précieuse pour l'apprentissage : on voit à la fois ce qu'on écrit et ce que Typst produit.

L'intégrale de Gauss — résultat fondamental en probabilité et théorie des fonctions, qui établit que l'aire sous la courbe en cloche e^{-x^2} vaut exactement $\sqrt{\pi}$:

```
$$\int_{-\infty}^{+\infty} e^{-x^2} dx = \sqrt{\pi}$$
```

$$\int_{-\infty}^{+\infty} e^{-x^2} dx = \sqrt{\pi}$$

où $x \in \mathbb{R}$ est la variable d'intégration et e^{-x^2} est la fonction gaussienne non normalisée. C'est cette identité qui fonde, après normalisation par $\sqrt{2\pi}\sigma$, la densité de la loi normale.

La Transformée de Fourier (décomposition) – extrait les fréquences d'un signal temporel pour révéler son contenu spectral :

$$\hat{f}(\xi) = \int_{-\infty}^{+\infty} f(t) e^{-i 2\pi \xi t} dt$$

où $f(t)$ est le signal d'origine, $\hat{f}(\xi)$ sa transformée, et $e^{-i2\pi\xi t}$ le noyau exponentiel complexe qui projette le signal sur chaque fréquence. La constante i désigne l'unité imaginaire ($i^2 = -1$).

La Transformée de Fourier inverse (reconstruction) – opération réciproque qui reconstruit le signal temporel à partir de son spectre :

$$f(t) = \int_{-\infty}^{+\infty} \hat{f}(\xi) e^{i 2\pi \xi t} d\xi$$

où les rôles de t et ξ sont inversés par rapport à la transformée directe : on intègre sur la variable de fréquence ξ , et le signe du noyau exponentiel passe de $-i$ à $+i$ (ce qui assure la réversibilité de l'opération).

2.3 Les chunks de code R

Un **chunk** est un bloc de code exécutable intégré au document. Le chunk ci-dessous génère la séquence des entiers de 1 à 20 :

```
seq(1, 20) # Fonction générique : séquence de 1 à 20 par pas de 1
```

```
#> [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
```

```
1:20 # Opérateur : notation compacte équivalente
```

```
#> [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
```

Les années de Coupe du Monde de football entre 1970 et 2026 :

```
seq(1970, 2026, by = 4)
```

```
#> [1] 1970 1974 1978 1982 1986 1990 1994 1998 2002 2006 2010 2014 2018 2022 2026
```


💡 À retenir

La fonction `seq(de, à, by = pas)` est l'une des plus utiles de R. On peut aussi écrire `1:20` pour une séquence d'entiers consécutifs — c'est la notation compacte équivalente à `seq(1, 20, by = 1)`.

2.4 Références bibliographiques dans Quarto

Quarto gère nativement les bibliographies grâce à **BibTeX**. Il suffit d'indiquer le fichier `.bib` dans l'entête YAML et de citer les références avec la syntaxe `@cle_bib`.

Par exemple, on peut citer l'article fondateur du Tidyverse : H. Wickham *et al.* [3].

3 Le Tidyverse

3.1 Présentation

Le **Tidyverse** est un écosystème de packages R conçu pour la *data science*. Tous ces packages partagent une même philosophie, une même grammaire et des structures de données communes (les *tibbles*).

i Analogie pédagogique

Le Tidyverse, c'est comme une boîte à outils bien organisée : chaque outil a un rôle précis, ils fonctionnent tous ensemble, et on apprend une fois la logique commune plutôt que de mémoriser des syntaxes disparates.

Parmi les packages les plus utilisés :

- `dplyr` : manipulation des données (filtrer, sélectionner, grouper, résumer)
- `ggplot2` : visualisation graphique déclarative
- `tidyr` : restructuration des données (pivot, imbrication)
- `readr` : lecture de fichiers CSV et délimités
- `lubridate` : traitement des dates et heures
- `stringr` : manipulation des chaînes de caractères

3.2 Manipulation d'un data frame en R de base

```
dfcars <- mtcars
```

```
dfcars$mpg      # Affiche les 32 valeurs de mpg (miles par gallon)
```

```
#> [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4  
#> [16] 10.4 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7  
#> [31] 15.0 21.4
```

```
dfcars[, 1] # Sélection de la 1ère colonne (toutes les lignes)
```

```
#> [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4  
#> [16] 10.4 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7  
#> [31] 15.0 21.4
```

```
dfcars[1, ] # Sélection de la 1ère ligne (toutes les colonnes)
```

```
#>      mpg cyl disp  hp drat   wt  qsec vs am gear carb  
#> Mazda RX4    21   6  160 110  3.9 2.62 16.46  0  1    4    4
```

```
dfcars[1, 1] # Valeur unique : 1ère colonne, 1ère ligne
```

```
#> [1] 21
```

3.3 Manipulations avec le Tidyverse et le pipe |>

Le **pipe natif** |> (disponible depuis R 4.1) permet de chaîner les opérations de gauche à droite :

```
dfcars |> dplyr::select(mpg) # Sélection d'une colonne par son nom
```

```
#>      mpg  
#> Mazda RX4      21.0  
#> Mazda RX4 Wag  21.0  
#> Datsun 710      22.8  
#> Hornet 4 Drive  21.4  
#> Hornet Sportabout 18.7  
#> Valiant         18.1  
#> Duster 360      14.3  
#> Merc 240D       24.4  
#> Merc 230        22.8  
#> Merc 280        19.2  
#> Merc 280C       17.8  
#> Merc 450SE      16.4  
#> Merc 450SL      17.3  
#> Merc 450SLC     15.2  
#> Cadillac Fleetwood 10.4  
#> Lincoln Continental 10.4  
#> Chrysler Imperial 14.7  
#> Fiat 128        32.4  
#> Honda Civic     30.4  
#> Toyota Corolla  33.9  
#> Toyota Corona   21.5  
#> Dodge Challenger 15.5
```

```
#> AMC Javelin      15.2
#> Camaro Z28       13.3
#> Pontiac Firebird  19.2
#> Fiat X1-9        27.3
#> Porsche 914-2    26.0
#> Lotus Europa     30.4
#> Ford Pantera L   15.8
#> Ferrari Dino     19.7
#> Maserati Bora    15.0
#> Volvo 142E      21.4
```

```
select(dfcars, mpg)
```

```
# Même résultat, sans le pipe
```

```
#>      mpg
#> Mazda RX4      21.0
#> Mazda RX4 Wag  21.0
#> Datsun 710      22.8
#> Hornet 4 Drive  21.4
#> Hornet Sportabout 18.7
#> Valiant        18.1
#> Duster 360     14.3
#> Merc 240D      24.4
#> Merc 230       22.8
#> Merc 280       19.2
#> Merc 280C      17.8
#> Merc 450SE     16.4
#> Merc 450SL     17.3
#> Merc 450SLC    15.2
#> Cadillac Fleetwood 10.4
#> Lincoln Continental 10.4
#> Chrysler Imperial 14.7
#> Fiat 128       32.4
#> Honda Civic    30.4
#> Toyota Corolla 33.9
#> Toyota Corona  21.5
#> Dodge Challenger 15.5
#> AMC Javelin    15.2
#> Camaro Z28     13.3
#> Pontiac Firebird 19.2
#> Fiat X1-9      27.3
#> Porsche 914-2  26.0
#> Lotus Europa   30.4
#> Ford Pantera L 15.8
#> Ferrari Dino   19.7
#> Maserati Bora  15.0
#> Volvo 142E     21.4
```

```
dfcars |> dplyr::select(7:11) # Sélection des colonnes 7 à 11 (plage)
```

```
#>           qsec vs am gear carb
#> Mazda RX4      16.46  0  1    4    4
#> Mazda RX4 Wag  17.02  0  1    4    4
#> Datsun 710      18.61  1  1    4    1
#> Hornet 4 Drive  19.44  1  0    3    1
#> Hornet Sportabout 17.02  0  0    3    2
#> Valiant        20.22  1  0    3    1
#> Duster 360     15.84  0  0    3    4
#> Merc 240D      20.00  1  0    4    2
#> Merc 230       22.90  1  0    4    2
#> Merc 280       18.30  1  0    4    4
#> Merc 280C      18.90  1  0    4    4
#> Merc 450SE     17.40  0  0    3    3
#> Merc 450SL     17.60  0  0    3    3
#> Merc 450SLC    18.00  0  0    3    3
#> Cadillac Fleetwood 17.98  0  0    3    4
#> Lincoln Continental 17.82  0  0    3    4
#> Chrysler Imperial 17.42  0  0    3    4
#> Fiat 128       19.47  1  1    4    1
#> Honda Civic    18.52  1  1    4    2
#> Toyota Corolla 19.90  1  1    4    1
#> Toyota Corona 20.01  1  0    3    1
#> Dodge Challenger 16.87  0  0    3    2
#> AMC Javelin    17.30  0  0    3    2
#> Camaro Z28     15.41  0  0    3    4
#> Pontiac Firebird 17.05  0  0    3    2
#> Fiat X1-9      18.90  1  1    4    1
#> Porsche 914-2  16.70  0  1    5    2
#> Lotus Europa   16.90  1  1    5    2
#> Ford Pantera L 14.50  0  1    5    4
#> Ferrari Dino   15.50  0  1    5    6
#> Maserati Bora   14.60  0  1    5    8
#> Volvo 142E     18.60  1  1    4    2
```

3.4 Statistiques descriptives avec le Tidyverse

```
stat_des <- dfcars |>
  select(mpg) |>
  summarise(
    Moyenne      = mean(mpg),
    `Écart-type` = sd(mpg),
    Minimum      = min(mpg),
    Maximum      = max(mpg)
  )
```

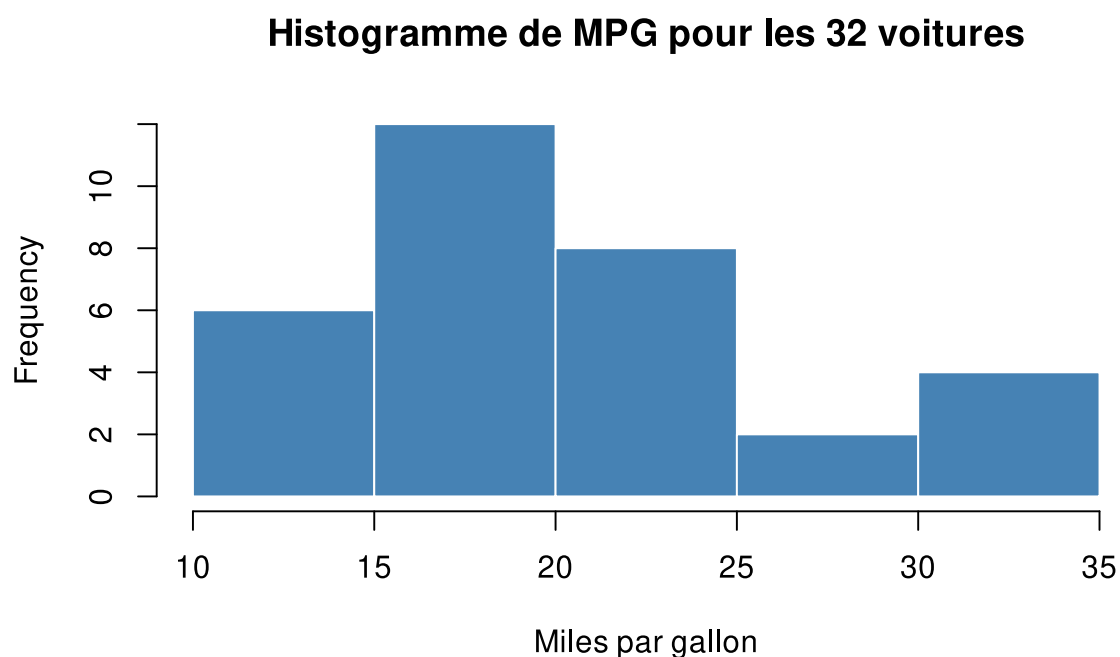
```
stat_des |>
  tt(digits = 2,
     caption = "Statistiques descriptives de la variable mpg (mtcars)")
```

Moyenne	Écart-type	Minimum	Maximum
20	6	10	34

4 Introduction à la visualisation avec R

4.1 Histogramme de base (R natif)

```
dfcars$mpg |> hist(main = "Histogramme de MPG pour les 32 voitures",
  xlab = "Miles par gallon", col = "steelblue", border = "white")
```



On peut ensuite filtrer les voitures très économes (30 à 35 mpg) :

```
dfcars |> filter(mpg >= 30 & mpg <= 35)
```

```
#>      mpg cyl disp  hp drat   wt  qsec vs am gear carb
#> Fiat 128  32.4   4  78.7  66 4.08 2.200 19.47  1  1    4    1
#> Honda Civic 30.4   4  75.7  52 4.93 1.615 18.52  1  1    4    2
#> Toyota Corolla 33.9   4  71.1  65 4.22 1.835 19.90  1  1    4    1
#> Lotus Europa 30.4   4  95.1 113 3.77 1.513 16.90  1  1    5    2
```

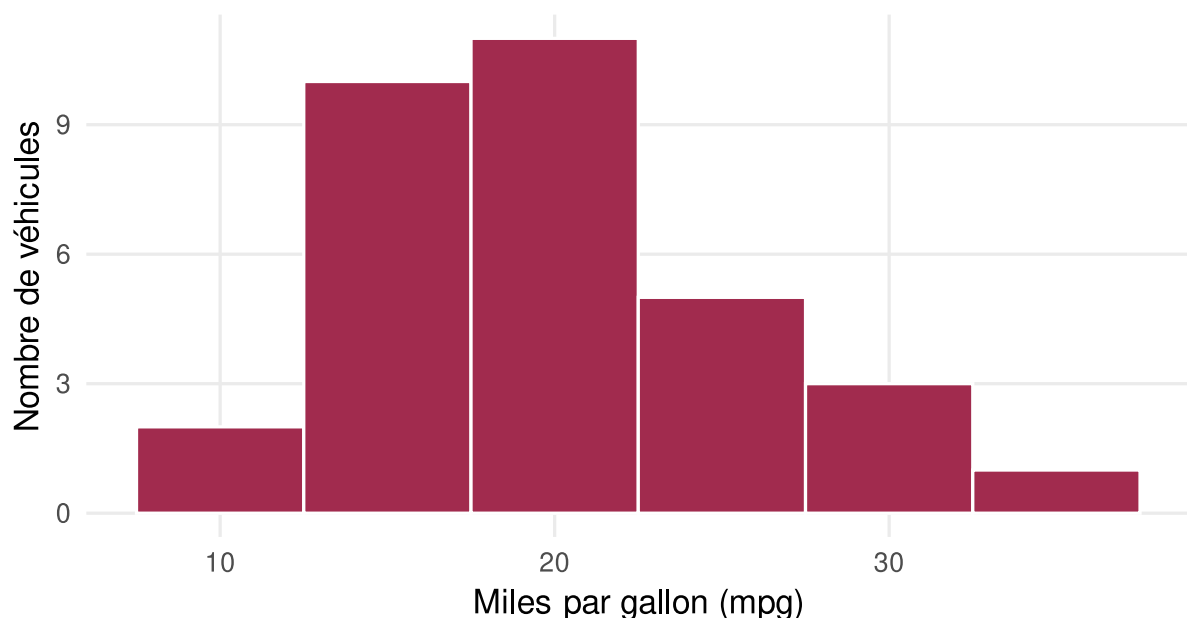
4.2 Histogramme avec ggplot2

ggplot2 offre une grammaire graphique déclarative : on décrit *ce que l'on veut voir*, et non *comment le dessiner*.

```
dfcars |>
  ggplot(aes(x = mpg)) +
  geom_histogram(fill = "#a12b4e", binwidth = 5, color = "white") +
  labs(
    title    = "Distribution de la consommation des 32 véhicules",
    subtitle = "Données : jeu intégré mtcars",
    x        = "Miles par gallon (mpg)",
    y        = "Nombre de véhicules"
  )
```

Distribution de la consommation des 32 véhicules

Données : jeu intégré mtcars



⚠ Point d'attention

Le thème ggplot2 est configuré globalement dans le chunk setup via `theme_set()`. Il n'est pas nécessaire d'ajouter `+ theme_minimal()` à chaque graphique — le thème s'applique automatiquement à tous les plots du document.

Partie II — Projet Keno : analyse des données FDJ

5 Projet Keno — Analyse des données FDJ

5.1 Présentation du Keno

Le **Keno** est un jeu de tirage proposé par la Française des Jeux (FDJ). Dans la version étudiée ici, **16 boules** sont tirées parmi 56 numérotées de 1 à 56. Le joueur coche jusqu'à 10 numéros, et ses gains dépendent du nombre de numéros cochés figurant parmi les boules tirées.

5.2 Chargement des données

```
keno_202511 <- read_delim(  
  "keno_202511.csv",  
  delim           = ";",  
  escape_double   = FALSE,  
  col_types       = cols(  
    date_de_forclusion = col_skip(),  
    ...23             = col_skip()  
  ),  
  trim_ws         = TRUE  
)  
  
keno_202511 <- keno_202511 |> select(-c(1, 19, 20, 21))  
  
names(keno_202511)
```

```
#> [1] "date_de_tirage" "boule1"      "boule2"      "boule3"  
#> [5] "boule4"         "boule5"      "boule6"      "boule7"  
#> [9] "boule8"         "boule9"      "boule10"     "boule11"  
#> [13] "boule12"        "boule13"     "boule14"     "boule15"  
#> [17] "boule16"
```

5.3 Pivot longer : restructuration des données

5.3.1 Le concept de pivot

Les données Keno arrivent au format **large** (wide) : chaque tirage est une ligne, avec 16 colonnes `boule1`, `boule2`, ..., `boule16`. Pour les analyses statistiques, il est plus commode d'avoir un format **long** : une ligne par boule tirée.


```
table4a <- tibble(
  country = c("A", "B", "C"),
  `1999` = c(700, 37000, 212000),
  `2000` = c(2000, 80000, 213000)
)
table4a
```

```
#> # A tibble: 3 × 3
#>   country `1999` `2000`
#>   <chr>    <dbl> <dbl>
#> 1 A         700    2000
#> 2 B        37000  80000
#> 3 C       212000 213000
```

```
table4a |>
  pivot_longer(
    cols = 2:3,
    names_to = "annee",
    values_to = "cas"
  )
```

```
#> # A tibble: 6 × 3
#>   country annee    cas
#>   <chr>   <chr> <dbl>
#> 1 A      1999     700
#> 2 A      2000    2000
#> 3 B      1999   37000
#> 4 B      2000   80000
#> 5 C      1999  212000
#> 6 C      2000  213000
```

5.3.2 Application au Keno

```
keno_longer <- keno_202511 |>
  pivot_longer(
    cols = 2:17,
    names_to = "Boule",
    names_prefix = "boule",
    values_to = "Num_boule"
  ) |>
  mutate(
    Boule = as.integer(Boule),
    Num_boule = as.integer(Num_boule)
  )

glimpse(keno_longer)
```

```
#> Rows: 1,600
#> Columns: 3
#> $ date_de_tirage <chr> "10/02/2026", "10/02/2026", "10/02/2026", "10/02/2026", ...
#> $ Boule <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, ...
#> $ Num_boule <int> 12, 13, 17, 20, 22, 23, 26, 27, 39, 42, 45, 48, 50, 52, ...
```

5.4 Tableau de fréquence des boules

```
freq_boules <- keno_longer |>
  dplyr::count(Num_boule, name = "frequence")

freq_boules |> filter(frequence == min(frequence))
```

```
#> # A tibble: 1 × 2
#>   Num_boule frequence
#>   <int>     <int>
#> 1         5        17
```

5.5 Formatage des dates

```
keno_202511 <- keno_202511 |>
  mutate(date_de_tirage = as.Date(date_de_tirage, format = "%d/%m/%Y"))

keno_202511 |>
  summarise(
    date_min = format(min(date_de_tirage), "%d/%m/%Y"),
    date_max = format(max(date_de_tirage), "%d/%m/%Y")
  )
```

```
#> # A tibble: 1 × 2
#>   date_min date_max
#>   <chr>    <chr>
#> 1 03/11/2025 10/02/2026
```

5.6 Palmarès des numéros — style FDJ

Ce tableau s'inspire du **palmarès des numéros** du site FDJ. Pour chaque numéro de 1 à 56, on affiche le nombre de sorties, le pourcentage de présence et la date de dernière sortie. La boîte ci-dessous précise la formule de calcul utilisée et fournit la référence théorique pour interpréter correctement les valeurs affichées.

i Rappel théorique

Calcul du pourcentage de présence. Pour chaque numéro $i \in \{1, \dots, 56\}$, on note S_i le nombre de tirages dans lesquels il est sorti. Avec N tirages historiques disponibles, le pourcentage de présence du numéro i est défini par :

$$\text{Pct}_i = \frac{S_i}{N} \times 100$$

où S_i est le nombre de sorties du numéro i et N le nombre total de tirages. Cette formule est strictement **identique au calcul officiel publié par la FDJ** sur sa page de statistiques Keno.

Référence théorique. À chaque tirage, 16 boules sont tirées parmi 56. La probabilité qu'un numéro donné figure dans un tirage est donc :

$$p_{\text{théo}} = \frac{16}{56} \approx 0,2857 = \mathbf{28,57\%}$$

Lecture du tableau. Un numéro affichant un pourcentage proche de **28,57 %** est conforme à la loi théorique. Un écart vers le haut indique un numéro *sur-représenté* sur la période, vers le bas un numéro *sous-représenté*. Ces écarts relèvent de la fluctuation d'échantillonnage — ils n'ont aucune valeur prédictive pour les tirages futurs.

```
df_keno <- read_delim("keno_202511.csv", delim = ";", show_col_types = FALSE)

## — Correction robuste du décompte des tirages —————
total_tirages <- df_keno |>
  mutate(date_dt = dmy(date_de_tirage)) |>
  filter(!is.na(date_dt)) |>
  distinct(date_dt) |>
  nrow()

message("Nombre de tirages distincts retenus : ", total_tirages)

freq_boules_fdj <- df_keno |>
  pivot_longer(
    cols = matches("^boule[0-9]+$"),
    names_to = "position",
    values_to = "Num_boule"
  ) |>
  mutate(
    date_dt = dmy(date_de_tirage),
    Num_boule = as.integer(Num_boule)
  ) |>
  filter(!is.na(date_dt), !is.na(Num_boule)) |>
  group_by(Num_boule) |>
  summarise(
    Sorties = n(),
```

```

Pct      = (n() / total_tirages) * 100,
Derniere = max(date_dt, na.rm = TRUE)
) |>
arrange(Num_boule)

freq_boules_fdj |>
  mutate(
    Pct      = paste0(sprintf("%.2f", Pct), " %"),
    Derniere = format(Derniere, "%d/%m/%y")
  ) |>
  select(
    `Numéros`      = Num_boule,
    `Nombre de sorties` = Sorties,
    `% de sorties`   = Pct,
    `Dernière sortie` = Derniere
  ) |>
  tt(caption = paste0(
    "Palmarès des numéros Keno FDJ. ",
    "* Pourcentage calculé sur N = ", total_tirages,
    " tirages valides distincts. Formule : sorties / N x 100."
  )) |>
  style_tt(align = "lccc")

```

Numéros	Nombre de sorties	% de sorties	Dernière sortie
1	30	30.00 %	09/02/26
2	26	26.00 %	08/02/26
3	28	28.00 %	09/02/26
4	32	32.00 %	09/02/26
5	17	17.00 %	02/02/26
6	21	21.00 %	05/02/26
7	37	37.00 %	08/02/26
8	23	23.00 %	04/02/26
9	33	33.00 %	09/02/26
10	30	30.00 %	02/02/26
11	28	28.00 %	09/02/26
12	28	28.00 %	10/02/26
13	38	38.00 %	10/02/26
14	35	35.00 %	09/02/26
15	21	21.00 %	09/02/26
16	23	23.00 %	06/02/26
17	33	33.00 %	10/02/26

Numéros	Nombre de sorties	% de sorties	Dernière sortie
18	19	19.00 %	04/02/26
19	38	38.00 %	08/02/26
20	24	24.00 %	10/02/26
21	29	29.00 %	08/02/26
22	20	20.00 %	10/02/26
23	29	29.00 %	10/02/26
24	29	29.00 %	02/02/26
25	27	27.00 %	09/02/26
26	28	28.00 %	10/02/26
27	36	36.00 %	10/02/26
28	33	33.00 %	05/02/26
29	26	26.00 %	08/02/26
30	35	35.00 %	06/02/26
31	29	29.00 %	09/02/26
32	24	24.00 %	30/01/26
33	24	24.00 %	28/01/26
34	25	25.00 %	02/02/26
35	29	29.00 %	08/02/26
36	24	24.00 %	09/02/26
37	29	29.00 %	03/02/26
38	31	31.00 %	05/02/26
39	27	27.00 %	10/02/26
40	27	27.00 %	08/02/26
41	38	38.00 %	06/02/26
42	30	30.00 %	10/02/26
43	25	25.00 %	08/02/26
44	30	30.00 %	08/02/26
45	25	25.00 %	10/02/26
46	32	32.00 %	09/02/26
47	31	31.00 %	02/02/26
48	30	30.00 %	10/02/26
49	32	32.00 %	07/02/26
50	30	30.00 %	10/02/26
51	30	30.00 %	08/02/26

Numéros	Nombre de sorties	% de sorties	Dernière sortie
52	29	29.00 %	10/02/26
53	32	32.00 %	09/02/26
54	24	24.00 %	04/02/26
55	33	33.00 %	10/02/26
56	24	24.00 %	10/02/26

5.7 Statistiques sur le format long

```
keno_long <- keno_202511 |>
  pivot_longer(
    cols          = boule1:boule16,
    names_to      = "boule",
    names_prefix  = "boule",
    names_transform = list(boule = as.integer),
    values_to     = "numero"
  )

table_freq <- keno_long |>
  dplyr::count(numero, name = "Nombre de sorties") |>
  complete(numero = 1:56, fill = list(`Nombre de sorties` = 0)) |>
  arrange(numero)

table_freq |>
  setNames(c("Numéro", "Nombre de sorties")) |>
  tt(caption = "Fréquence d'apparition des numéros (Keno – données FDJ)")
```

Numéro	Nombre de sorties
1	30
2	26
3	28
4	32
5	17
6	21
7	37
8	23
9	33
10	30
11	28
12	28
13	38

Numéro	Nombre de sorties
14	35
15	21
16	23
17	33
18	19
19	38
20	24
21	29
22	20
23	29
24	29
25	27
26	28
27	36
28	33
29	26
30	35
31	29
32	24
33	24
34	25
35	29
36	24
37	29
38	31
39	27
40	27
41	38
42	30
43	25
44	30
45	25
46	32
47	31

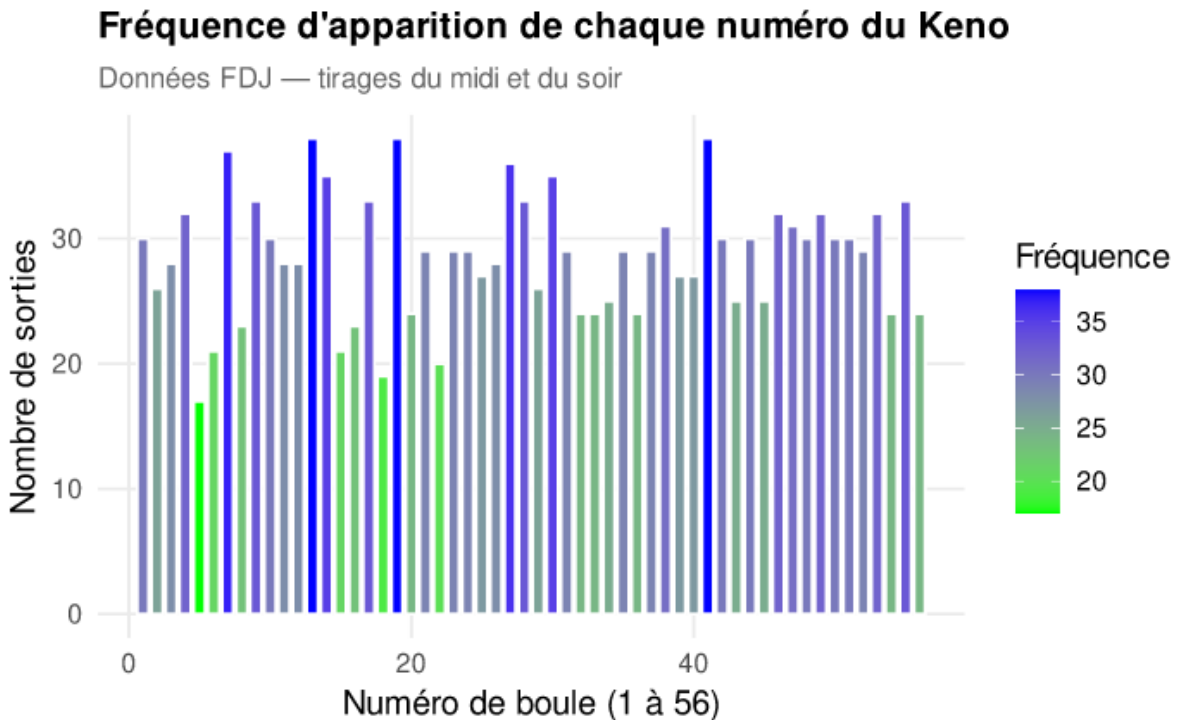
Numéro	Nombre de sorties
48	30
49	32
50	30
51	30
52	29
53	32
54	24
55	33
56	24

6 Visualisations graphiques du Keno

6.1 Histogramme des fréquences

```
freq_boules_plot <- keno_long |>
  dplyr::count(numero, name = "frequence") |>
  complete(numero = 1:56, fill = list(frequence = 0)) |>
  arrange(numero)

freq_boules_plot |>
  ggplot(aes(x = numero, y = frequence, fill = frequence)) +
  geom_bar(stat = "identity", color = "white", width = 0.8) +
  scale_fill_gradient(low = "green", high = "blue") +
  labs(
    title = "Fréquence d'apparition de chaque numéro du Keno",
    subtitle = "Données FDJ – tirages du midi et du soir",
    x = "Numéro de boule (1 à 56)",
    y = "Nombre de sorties",
    fill = "Fréquence"
  ) +
  theme(legend.position = "right")
```

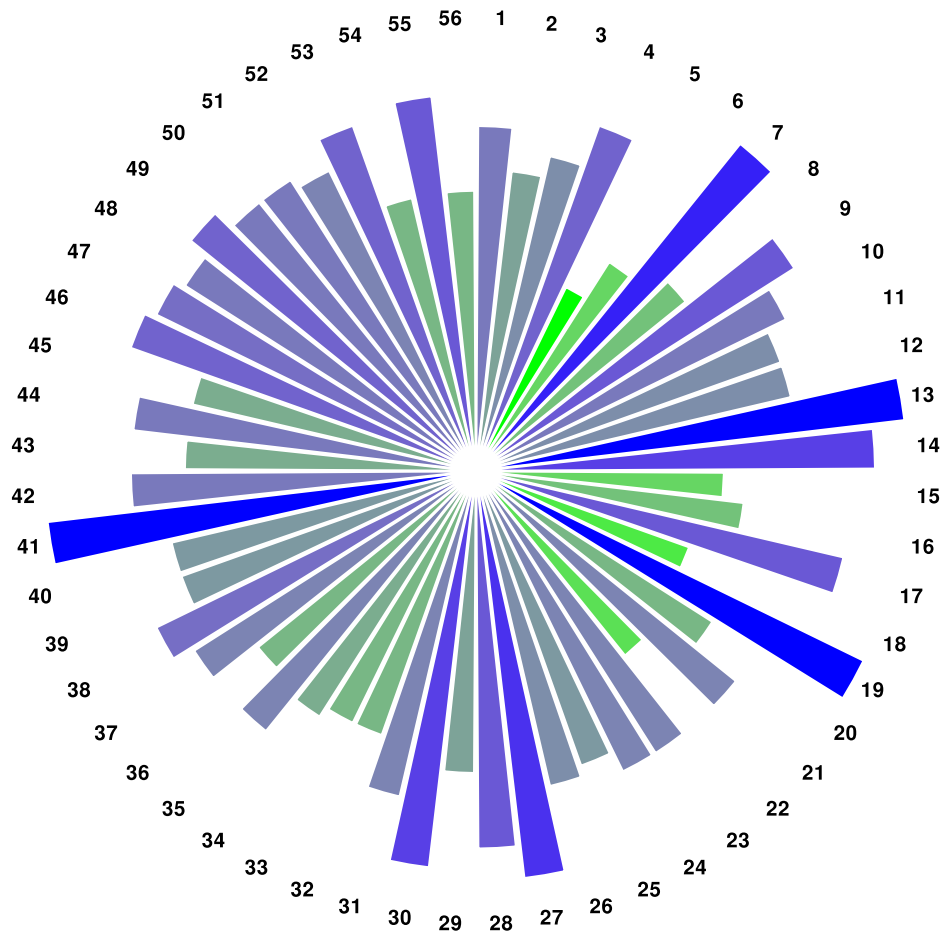
6.2 Graphique circulaire (diagramme en rose)

Ce graphique polaire enroule les 56 numéros autour d'un axe circulaire, permettant de visualiser d'un coup d'œil l'uniformité des tirages :

```
freq_boules_plot |>
  ggplot(aes(
    x = reorder(as.factor(numero), as.numeric(numero)),
    y = frequency,
    fill = frequency
  )) +
  geom_bar(stat = "identity", show.legend = FALSE, color = "white") +
  coord_polar(theta = "x", clip = "off") +
  geom_text(aes(y = 40, label = numero), color = "black", size = 3, fontface =
"bold") +
  ylim(-2, max(freq_boules_plot$frequency) + 2) +
  scale_fill_gradient(low = "green", high = "blue") +
  labs(title = "Répartition circulaire des fréquences",
    subtitle = "Chaque secteur représente un numéro (1 à 56)") +
  theme_void() +
  theme(plot.title = element_text(face = "bold", hjust = 0.5, size = 11),
    plot.subtitle = element_text(hjust = 0.5, color = "grey40", size = 9),
    plot.margin = margin(0, 0, 0, 0, "pt"),
    aspect.ratio = 1)
```

Répartition circulaire des fréquences

Chaque secteur représente un numéro (1 à 56)



7 Modélisation probabiliste du Keno

7.1 La loi hypergéométrique

i Rappel théorique

Le Keno est un tirage **sans remise** : les boules ne sont pas remplacées dans l'urne. La distribution de probabilité adaptée est la **loi hypergéométrique**.

Code LaTeX de la formule :

```

$$P(X = x) = \frac{\binom{K}{x} \binom{N-K}{n-x}}{\binom{N}{n}}$$

```

$$P(X = x) = \frac{\binom{K}{x} \binom{N-K}{n-x}}{\binom{N}{n}}$$

avec : $N = 56$ (taille de l'urne), $K = 16$ (boules tirées par le Keno), $n = 10$ (numéros cochés par le joueur), x (nombre de bons numéros).

La fonction R correspondante est `dhyper(x, m, n, k)` où m est le nombre de boules « succès » dans l'urne, n le nombre de boules « échec », et k le nombre de tirages.

7.2 Calcul des probabilités de gain

```
tibble(  
  x      = 0:10,  
  `p(x)` = dhyper(0:10, m = 16, n = 40, k = 10),  
  chance = round(1 / `p(x)`)  
)
```

```
#> # A tibble: 11 × 3  
#>       x      `p(x)`  chance  
#>   <int>    <dbl>    <dbl>  
#> 1     0 0.0238      42  
#> 2     1 0.123       8  
#> 3     2 0.259       4  
#> 4     3 0.293       3  
#> 5     4 0.196       5  
#> 6     5 0.0807     12  
#> 7     6 0.0206     49  
#> 8     7 0.00317    315  
#> 9     8 0.000282   3547  
#> 10    9 0.0000129  77813  
#> 11   10 0.000000225 4446435
```

7.3 Tableau des gains officiels FDJ

Le tableau ci-dessous croise les probabilités calculées avec les **gains officiels** publiés par la FDJ (pour une mise de 1 €).

```
tableau_gains <- tibble(
  x      = 0:10,
  `p(x)` = dhyper(x, m = 16, n = 40, k = 10)
) |>
  arrange(desc(x)) |>
  filter(!x %in% 1:4) |>
  mutate(`g(x) en €` = c(200000, 2000, 150, 15, 5, 2, 2), .after = x)

tableau_gains |>
  setNames(c("Bonnes boules (x)", "Gain g(x) en €", "Probabilité p(x)")) |>
  tt(digits = 6,
     caption = "Gains officiels FDJ et probabilités associées (mise de 1 €, 10
numéros cochés)") |>
  style_tt(i = 1, bold = TRUE, color = "white", background = "#a12b4e")
```

Bonnes boules (x)	Gain g(x) en €	Probabilité p(x)
10	200000	0.000000224899
9	2000	0.000012851387
8	150	0.000281927303
7	15	0.003174292599
6	5	0.020553544581
5	2	0.080719375083
0	2	0.023805973614

8 Référence officielle FDJ : tableau complet des gains

8.1 Les tableaux de gains officiels

Les probabilités et gains présentés dans cette section sont calculés à partir des règles officielles du Keno FDJ. Les sources officielles sont consultables en ligne :

- **Probabilités et chances de gagner** : [fdj.fr — Quelles sont les chances de gagner au Kéno ?](https://fdj.fr/Quelles-sont-les-chances-de-gagner-au-Keno/)
- **Statistiques officielles des tirages** : [fdj.fr — Kéno Statistiques](https://fdj.fr/Keno-Statistiques)

⚠ Note sur le modèle utilisé

Les tableaux de gains de ce document sont calculés sur le modèle **16 boules tirées parmi 56**, qui correspond exactement aux données historiques disponibles (keno_202511.csv). Ce modèle a été en vigueur jusqu'en 2020 ; la version actuelle du Keno FDJ tire 20 boules parmi 70. Les **gains officiels** (montants en euros) sont identiques entre les deux versions, mais les **probabilités** calculées par `dhyper()` diffèrent. Toutes les analyses de ce document restent cohérentes en se basant systématiquement sur le modèle 16/56.

COMMENT CONNAITRE VOS GAINS ?				
Consultez le tableau des gains (mise 1 € ou 10 €) ci-dessous. Pour une mise de 2 €, 3 € ou 5 €, les gains correspondant à une mise de 1 € sont respectivement multipliés par 2, 3 ou 5.				
N° choisis par grille	N° trouvés par grille	Vos chances de gagner	Gain pour une mise de	
			1 €	10 €
10	10	1 sur 2 147 181	10 000 €/an À VIE ou 200 000 € CASH	100 000 €/an À VIE ou 2 000 000 € CASH
	9	1 sur 47 238	2 000 €	20 000 €
	8	1 sur 2 571	100 €	1 000 €
	7	1 sur 261	10 €	100 €
	6	1 sur 44	5 €	50 €
	5	1 sur 12	2 €	20 €
	0	1 sur 39	2 €	20 €
9	9	1 sur 387 197	2 000 €/an À VIE ou 40 000 € CASH	20 000 €/an À VIE ou 400 000 € CASH
	8	1 sur 10 325	100 €	1 000 €
	7	1 sur 685	20 €	200 €
	6	1 sur 86	8 €	80 €
	5	1 sur 18	2 €	20 €
	4	1 sur 6	1 €	10 €
	0	1 sur 26	1 €	10 €
8	8	1 sur 74 941	8 000 €	80 000 €
	7	1 sur 2 436	100 €	1 000 €
	6	1 sur 199	30 €	300 €
	5	1 sur 31	5 €	50 €
	0	1 sur 18	2 €	20 €
7	7	1 sur 15 464	3 000 €	30 000 €
	6	1 sur 619	70 €	700 €
	5	1 sur 63	5 €	50 €
	4	1 sur 13	2 €	20 €
6	6	1 sur 3 383	900 €	9 000 €
	5	1 sur 169	30 €	300 €
	4	1 sur 22	3 €	30 €
5	5	1 sur 781	80 €	800 €
	4	1 sur 50	10 €	100 €
	3	1 sur 9	2 €	20 €
4	4	1 sur 189	50 €	500 €
	3	1 sur 16	5 €	50 €
3	3	1 sur 48	10 €	100 €
	2	1 sur 6	2 €	20 €
2	2	1 sur 13	6 €	60 €

Option Multiplicateur : si vous avez coché l'option, vos gains sont automatiquement multipliés par le N° Multiplicateur tiré au sort.

N° Multiplicateur (et chances de sortie) :

x 2 (1 chance sur 1,49)
 x 3 (1 chance sur 3,70)
 x 5 (1 chance sur 16,67)

Figure 1. — Tableau officiel FDJ — toutes grilles de 2 à 10 numéros (version 2020)

8.2 Construction de la table complète des gains FDJ

Nous reproduisons ici le tableau officiel FDJ pour **toutes les grilles** (de 2 à 10 numéros cochés), en calculant les probabilités théoriques avec `dhyper()` et en renseignant les gains officiels conformément au règlement FDJ — modèle 16 boules tirées sur 56, cohérent avec les données `keno_202511.csv`.

```
## — Paramètres du Keno FDJ — modèle des données historiques —————
N_keno <- 56 # taille de l'urne (version historique — données keno_202511.csv)
K_keno <- 16 # boules tirées par tirage (version historique)

## Table des gains officiels FDJ (mise 1 €) — source : article 8 du règlement
gains_fdj <- list(
  `10` = list(`10` = 200000, `9` = 2000, `8` = 150, `7` = 15,
              `6` = 5,    `5` = 2,    `0` = 2),
  `9` = list(`9` = 30000, `8` = 100, `7` = 25, `6` = 8,
             `5` = 2,    `4` = 1,    `0` = 2),
  `8` = list(`8` = 8000, `7` = 100, `6` = 30, `5` = 5, `0` = 2),
  `7` = list(`7` = 3000, `6` = 90, `5` = 5, `4` = 2),
  `6` = list(`6` = 900, `5` = 30, `4` = 3),
  `5` = list(`5` = 80, `4` = 10, `3` = 2),
  `4` = list(`4` = 70, `3` = 3),
  `3` = list(`3` = 10, `2` = 2),
  `2` = list(`2` = 6)
)

message("Table des gains FDJ chargée : ", length(gains_fdj), " grilles définies (2
à 10 numéros).")
```

```
df_gains_long <- purrr::imap_dfr(gains_fdj, function(gains_grille, n_coches_chr) {
  n <- as.integer(n_coches_chr)
  purrr::imap_dfr(gains_grille, function(gain, n_trouves_chr) {
    x <- as.integer(n_trouves_chr)
    p <- dhyper(x, m = K_keno, n = N_keno - K_keno, k = n)
    tibble(
      n_coches = n,
      n_trouves = x,
      prob = p,
      chance = if (p > 0) round(1 / p) else NA_real_,
      gain_leur = gain,
      gain_10eur = gain * 10
    )
  })
}) |>
  arrange(desc(n_coches), desc(n_trouves))

message("Tableau long construit : ", nrow(df_gains_long), " lignes.")
```

```
df_gains_long |>
  filter(n_coches == 10) |>
  select(
    `N° trouvés`      = n_trouves,
    `Probabilité`     = prob,
    `1 chance sur...` = chance,
    `Gain (1 €)`      = gain_leur,
    `Gain (10 €)`     = gain_10eur
  ) |>
  tt(digits = 7,
     caption = "Grille à 10 numéros cochés – probabilités et gains officiels FDJ
(16 boules sur 56, mise 1 euro)") |>
  style_tt(i = 1, bold = TRUE, color = "white", background = "#a12b4e") |>
  style_tt(j = 4, bold = TRUE)
```

N° trouvés	Probabilité	1 chance sur...	Gain (1 €)	Gain (10 €)
10	0.0000002248993	4446435	200000	2000000
9	0.000012851387	77813	2000	20000
8	0.0002819273032	3547	150	1500
7	0.0031742925994	315	15	150
6	0.0205535445812	49	5	50
5	0.0807193750826	12	2	20
0	0.0238059736139	42	2	20

```
tbl_complet <- df_gains_long |>
  mutate(
    prob_fmt = formatC(prob, format = "e", digits = 3),
    chance_fmt = formatC(chance, format = "fg", big.mark = " ",
                        flag = "#") |> stringr::str_trim(),
    gain_fmt = paste0(formatC(gain_leur, format = "fg", big.mark = " "), " €")
  ) |>
  select(
    `Grille`      = n_coches,
    `N° trouvés`  = n_trouves,
    `p(x)`        = prob_fmt,
    `1 chance sur` = chance_fmt,
    `Gain (1 €)`  = gain_fmt
  )

tbl_complet |>
  tt(caption = "Table complète des gains FDJ – toutes grilles de 2 à 10 numéros
cochés (16 boules tirées sur 56, mise 1 euro)") |>
  style_tt(j = 1:2, align = "c") |>
  style_tt(j = 3:5, align = "r")
```

Grille	N° trouvés	p(x)	1 chance sur	Gain (1 €)
10	10	2.249e-07	4 446 435	200 000 €
10	9	1.285e-05	77 813	2 000 €
10	8	2.819e-04	3 547	150 €
10	7	3.174e-03	315.0	15 €
10	6	2.055e-02	49.00	5 €
10	5	8.072e-02	12.00	2 €
10	0	2.381e-02	42.00	2 €
9	9	1.510e-06	662 235	30 000 €
9	8	6.795e-05	14 716	100 €
9	7	1.178e-03	849.0	25 €
9	6	1.044e-02	96.00	8 €
9	5	5.269e-02	19.00	2 €
9	4	1.581e-01	6.000	1 €
9	0	3.609e-02	28.00	2 €
8	8	9.060e-06	110 373	8 000 €
8	7	3.221e-04	3 104	100 €
8	6	4.397e-03	227.0	30 €
8	5	3.038e-02	33.00	5 €
8	0	5.414e-02	18.00	2 €
7	7	4.933e-05	20 272	3 000 €
7	6	1.381e-03	724.0	90 €
7	5	1.469e-02	68.00	5 €
7	4	7.753e-02	13.00	2 €
6	6	2.466e-04	4 055	900 €
6	5	5.381e-03	186.0	30 €
6	4	4.372e-02	23.00	3 €
5	5	1.144e-03	874.0	80 €
5	4	1.906e-02	52.00	10 €
5	3	1.144e-01	9.000	2 €
4	4	4.955e-03	202.0	70 €
4	3	6.099e-02	16.00	3 €
3	3	2.020e-02	49.00	10 €
3	2	1.732e-01	6.000	2 €
2	2	7.792e-02	13.00	6 €

8.3 Espérances de gain par grille

i Rappel théorique

Pour chaque grille (nombre de numéros cochés n), l'espérance de gain se calcule en sommant les gains pondérés par leurs probabilités, sur toutes les combinaisons gagnantes :

Code LaTeX de la formule :

```
$$E_n[G] = \sum_{x \in \mathcal{G}_n} g(x) \cdot p(x)$$
```

$$E_n[G] = \sum_{x \in \mathcal{G}_n} g(x) \cdot p(x)$$

où $\mathcal{G}_n$ est l'ensemble des valeurs de x donnant lieu à un gain pour la grille n .

```
df_esperances <- df_gains_long |>
  group_by(n_coches) |>
  summarise(
    esperance = sum(gain_leur * prob, na.rm = TRUE),
    .groups = "drop"
  ) |>
  mutate(
    trj_pct = round(esperance * 100, 2),
    perte_moy = round(1 - esperance, 4)
  ) |>
  arrange(desc(n_coches))

df_esperances |>
  select(
    `N° cochés` = n_coches,
    `Espérance E[G]` = esperance,
    `TRJ (%)` = trj_pct,
    `Perte moy. / tirage (EUR)` = perte_moy
  ) |>
  tt(digits = 4,
    caption = "Espérance de gain et taux de retour joueur (TRJ) par grille – Keno
FDJ (mise 1 euro)" |>
  style_tt(j = 3, bold = TRUE, background = "#fde8ef")
```

N° cochés	Espérance E[G]	TRJ (%)	Perte moy. / tirage (EUR)
10	0.4724	47.24	0.5276
9	0.5007	50.07	0.4993
8	0.4968	49.68	0.5032
7	0.5008	50.08	0.4992
6	0.5146	51.46	0.4854

N° cochés	Espérance E[G]	TRJ (%)	Perte moy. / tirage (EUR)
5	0.5108	51.08	0.4892
4	0.5298	52.98	0.4702
3	0.5483	54.83	0.4517
2	0.4675	46.75	0.5325

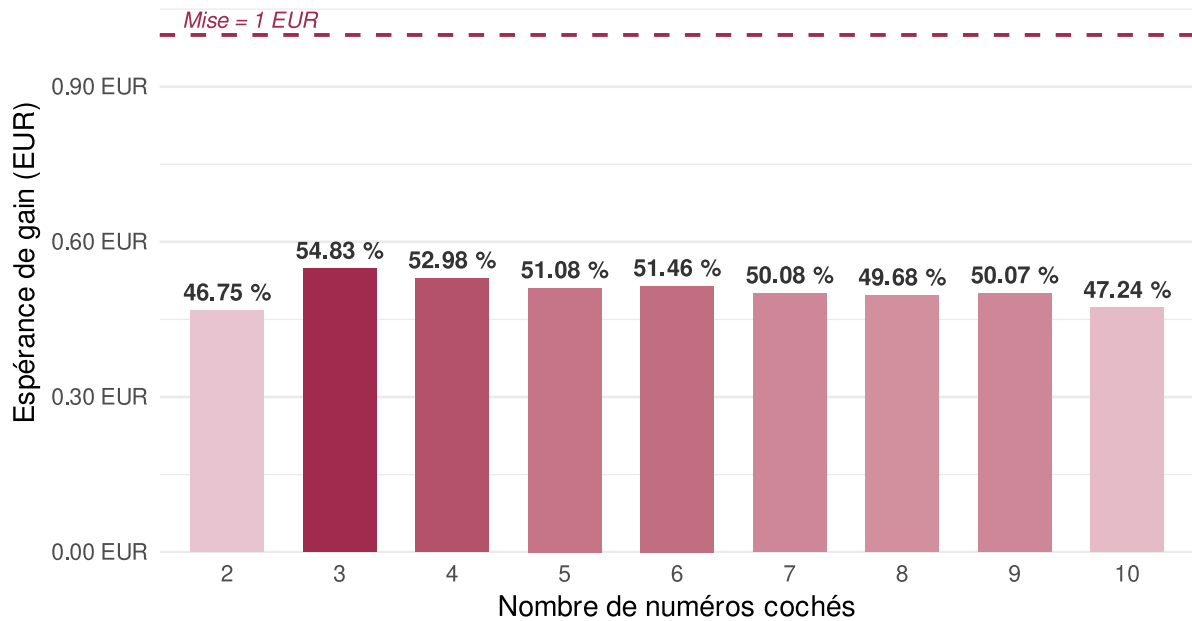
🔗 À retenir

Le **taux de retour joueur (TRJ)** est la fraction de la mise que le joueur récupère **en moyenne** sur un très grand nombre de parties. Un TRJ de 50 % signifie que pour 1 € joué, le joueur récupère en moyenne 0,50 € et perd donc 0,50 €. Ce taux est encadré réglementairement par l'ARJEL (Autorité de régulation des jeux en ligne).

```
df_esperances |>
  ggplot(aes(x = factor(n_coches), y = esperance)) +
  geom_col(aes(fill = esperance), width = 0.65, show.legend = FALSE) +
  geom_hline(yintercept = 1, linetype = "dashed",
             color = "#a12b4e", linewidth = 0.7) +
  geom_text(aes(label = paste0(trj_pct, " %")),
            vjust = -0.5, size = 3.2, fontface = "bold", color = "gray20") +
  scale_fill_gradient(low = "#e8c4d0", high = "#a12b4e") +
  scale_y_continuous(
    labels = scales::label_number(suffix = " EUR", accuracy = 0.01),
    limits = c(0, 1.10),
    expand = expansion(mult = c(0, 0.02))
  ) +
  annotate("text", x = 0.6, y = 1.03, label = "Mise = 1 EUR",
          color = "#a12b4e", size = 3, hjust = 0, fontface = "italic") +
  labs(
    title = "Espérance de gain selon le nombre de numéros cochés",
    subtitle = "Keno FDJ – mise de 1 euro – modèle 16/56",
    x = "Nombre de numéros cochés",
    y = "Espérance de gain (EUR)",
    caption = "Source : gains officiels FDJ. La ligne pointillée rouge
matérialise la mise de 1 EUR.\nEn dessous de cette ligne, le joueur perd en
espérance."
  ) +
  theme_minimal() +
  theme(panel.grid.major.x = element_blank())
```

Espérance de gain selon le nombre de numéros cochés

Keno FDJ — mise de 1 euro — modèle 16/56



Source : gains officiels FDJ. La ligne pointillée rouge matérialise la mise de 1 EUR. En dessous de cette ligne, le joueur perd en espérance.

Figure 2. – Espérance de gain selon le nombre de numéros cochés — Keno FDJ (mise 1 euro, modèle 16/56). Toutes les barres restent sous la ligne pointillée rouge, confirmant que l'espérance est toujours négative pour le joueur.

⚠ Lecture du graphique

Toutes les barres sont **sous la ligne pointillée rouge** (la mise de 1 €), confirmant que l'**espérance de gain est toujours négative pour le joueur**, quelle que soit la grille choisie. Le TRJ affiché au-dessus de chaque barre (en % de la mise) montre que la FDJ reverse entre 50 % et 60 % de la mise en gains — la différence constituant la marge opératrice.

9 Espérance de gain

9.1 Définition et formule

i Rappel théorique

L'**espérance mathématique** de gain est la valeur moyenne que le joueur peut espérer gagner par tirage. Elle se calcule comme la somme des gains pondérés par leurs probabilités :

Code LaTeX de la formule :

```
$$E[G] = \sum_{x=0}^{10} g(x) \cdot p(x)$$
```

$$E[G] = \sum_{x=0}^{10} g(x) \cdot p(x)$$

où $g(x)$ est le gain pour x bonnes boules, et $p(x)$ la probabilité d'en obtenir exactement x .

9.2 Calcul avec R

```
result <- tibble(  
  x      = 0:10,  
  `p(x)` = dhyper(x, m = 16, n = 40, k = 10)  
) |>  
  arrange(desc(x)) |>  
  filter(!x %in% 1:4) |>  
  mutate(`g(x)` = c(200000, 2000, 150, 15, 5, 2, 2), .after = x)  
  
esperance <- result |>  
  summarise(esperance = sum(`g(x)` * `p(x)`))  
  
esperance
```

```
#> # A tibble: 1 × 1  
#>   esperance  
#>   <dbl>  
#> 1      0.472
```

Interprétation : Pour une mise de 1 €, l'espérance de gain est d'environ **0.472 €**. Cela signifie que le joueur perd en moyenne 0.528 € à chaque tirage — ce qui traduit la marge de la FDJ.

⚠ Point d'attention

L'espérance de gain est inférieure à 1 euro (la mise). Cela signifie que le joueur perd en moyenne de l'argent à chaque tirage — c'est la **marge** intégrée par la FDJ dans le jeu. Ce résultat est fondamental en théorie des jeux et illustre pourquoi les jeux d'argent ne peuvent pas être des stratégies de gain à long terme.

10 Simulations de Monte-Carlo

10.1 Principe

La **simulation de Monte-Carlo** est une méthode numérique qui consiste à simuler un grand nombre de tirages aléatoires pour estimer des probabilités ou des distributions.

10.2 Premier exemple : simulation simple

```
set.seed(123)

numeros <- c()
for (t in 1:100) {
  numeros <- c(numeros, sample(1:56, 16, replace = FALSE))
}
table(numeros) |> min()
```

```
#> [1] 20
```

La même simulation, plus élégamment avec `replicate()` :

```
set.seed(123)
numeros_simu <- as.vector(replicate(100, sample(1:56, 16, replace = FALSE)))
table(numeros_simu) |> min()
```

```
#> [1] 20
```

10.3 Estimation d'une probabilité par Monte-Carlo

On répète l'expérience **N = 100 fois** pour estimer la probabilité qu'un numéro sorte 17 fois ou moins sur 100 tirages :

```
set.seed(123)
N <- 100

valmin <- replicate(N, {
  tirages <- replicate(100, sample(1:56, 16, replace = FALSE))
```

```

counts <- table(factor(as.vector(tirages), levels = 1:56))
min(counts)
})

cat("Probabilité estimée P(min ≤ 17) =", sum(valmin <= 17) / N)

```

```
#> Probabilité estimée P(min ≤ 17) = 0.26
```

10.4 Simulation de Monte-Carlo à grande échelle (1 000 itérations)

```

set.seed(123)
n_simulations <- 1000
tirages_par_serie <- 100

simuler_min <- function() {
  resultats <- replicate(tirages_par_serie, sample(1:56, 16, replace = FALSE))
  counts <- table(factor(resultats, levels = 1:56))
  return(min(counts))
}

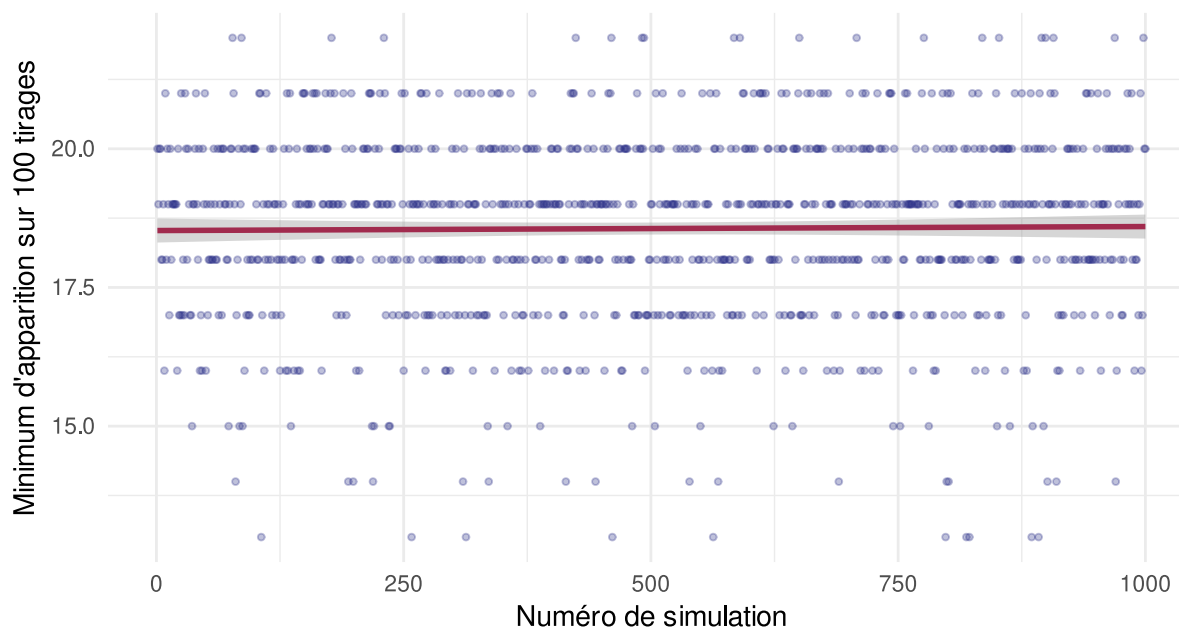
simus <- replicate(n_simulations, simuler_min())
df_simus <- data.frame(id = 1:n_simulations, val_min = simus)

ggplot(df_simus, aes(x = id, y = val_min)) +
  geom_point(alpha = 0.3, color = "#282D87", size = 0.9) +
  geom_smooth(method = "lm", color = "#a12b4e", se = TRUE) +
  labs(
    title = "Simulation de Monte-Carlo : fréquence minimale d'apparition",
    subtitle = "1 000 séries de 100 tirages – Keno (16 boules parmi 56)",
    x = "Numéro de simulation",
    y = "Minimum d'apparition sur 100 tirages"
  ) +
  theme_minimal()

```

Simulation de Monte-Carlo : fréquence minimale d'apparition

1 000 séries de 100 tirages — Keno (16 boules parmi 56)



```
seuil_critique <- mean(simus)
cat("En moyenne, sur 100 tirages, le numéro le moins sorti apparaît",
    round(seuil_critique, 1), "fois.")
```

```
#> En moyenne, sur 100 tirages, le numéro le moins sorti apparaît 18.6 fois.
```

💡 À retenir

La droite de régression (quasiment horizontale) confirme que le processus est **stationnaire** : le minimum d'apparition ne dérive pas au fil des simulations. C'est une propriété attendue d'un tirage véritablement aléatoire et uniforme.

Partie III — Méthodes de clustering

11 Introduction au clustering

Ce volet a été réalisé dans le cadre de la même unité d'enseignement **ESD113**. L'objectif principal est de reproduire et d'adapter le [code R](#) utilisé dans l'article de référence « **A Survey of Popular R Packages for Cluster Analysis** » [4].

11.1 Qu'est-ce que le clustering ?

Imaginez que l'on vous donne un grand sac contenant mille pièces de puzzle mélangées, issues de plusieurs boîtes différentes. Votre mission : les trier *sans voir les images* des boîtes d'origine. Naturellement, vous examinerez les couleurs, les formes de bordures, les textures — vous regrouperez les pièces qui *se ressemblent*. Sans le savoir, vous appliquerez l'algorithme mental à la base de tout clustering.

En statistique, le **clustering** (ou *classification non supervisée*) consiste à regrouper automatiquement des observations similaires **sans connaître à l'avance** les étiquettes ou catégories.

i Analogie pédagogique

Un algorithme de clustering appliqué à votre liste de courses pourrait découvrir, sans qu'on le lui dise, que vous achetez toujours ensemble des pâtes, de la sauce tomate et du parmesan — et créer un « groupe culinaire » reflétant ce comportement. C'est de la connaissance extraite automatiquement de l'observation brute.

11.2 Bref panorama historique

11.2.1 La petite (et longue) histoire de l'algorithme de Lloyd

Si l'on devait décerner le prix de la « patience algorithmique », Stuart Lloyd serait sans doute sur le podium. Imaginez la scène : nous sommes en **1957**, dans les prestigieux Laboratoires Bell. Entre deux tasses de café noir et des montagnes de tubes à vide, Lloyd pose les bases de ce qui deviendra le moteur de calcul le plus utilisé au monde pour le clustering : l'algorithme de quantification par moindres carrés. Il faudra attendre **1982** — soit 25 ans après sa conception initiale ! — pour que le travail de Lloyd 1982 soit enfin publié officiellement dans les *IEEE Transactions on Information Theory*. C'est un peu comme si quelqu'un inventait la roue en secret, la rangeait dans son garage, et attendait que tout le monde roule en carrosse pour enfin publier le brevet !

Quelques jalons historiques essentiels :

- **1894** — Karl Pearson 1894 utilise les premiers mélanges gaussiens pour analyser des populations de crabes dans la Baie de Naples. C'est l'acte de naissance des modèles de mélanges.
- **Années 1950–1960** — Les biologistes développent la *taxinomie numérique*. Les premières méthodes hiérarchiques émergent [7].

- **1963** — Joe Ward 1963 formalise la méthode de liaison de Ward.
- **1967** — James MacQueen 1967 publie l'article fondateur du **K-means**.
- **1977** — Dempster, Laird et Rubin 1977 formalisent l'algorithme **EM**.
- **2002** — Fraley et Raftery 2002 publient la référence sur Mclust.

11.3 Objectifs et plan du volet clustering

Ce volet guide à travers **trois grandes familles de méthodes**, du plus simple au plus sophistiqué :

1. Simulation de données structurées pour évaluer les algorithmes.
2. K-means, méthode du coude et ses **six variations d'implémentation**.
3. Classification hiérarchique ascendante et dendrogramme.
4. Modèles de mélanges gaussiens avec Mclust.

12 Préparation de l'environnement R pour le clustering

```
library(mvtnorm)      # Simulation de lois normales multivariées
library(mclust)       # Modèles de mélanges gaussiens + ARI
library(ggforce)      # Ellipses et hulls ggplot2
library(viridis)      # Palettes daltonisme-friendly
library(ggdendro)     # Dendrogrammes ggplot2
library(broom)        # Extraction standardisée de métriques
library(factoextra)   # Visualisation clustering

conflicts_prefer(dplyr::select)
conflicts_prefer(dplyr::filter)
conflicts_prefer(dplyr::lag)
```

13 Simulation des données : créer un laboratoire contrôlé

13.1 Pourquoi simuler ?

Évaluer une méthode de clustering sur des données réelles pose un problème fondamental : on ne connaît pas les *vrais* groupes, puisque c'est précisément ce qu'on cherche. La simulation résout ce problème en créant un monde artificiel où **la vérité est connue à l'avance**.

💡 À retenir

La simulation est le mannequin de crash-test de la statistique : on connaît exactement les forces appliquées, et on mesure comment chaque algorithme s'en sort.

13.2 Fondements mathématiques : la loi normale multivariée

13.2.1 La loi normale univariée

La loi normale (loi de Gauss) est la distribution de probabilité la plus fondamentale en statistique [12].

Sa densité est :

```
$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \quad x \in \mathbb{R}$$
```

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \quad x \in \mathbb{R}$$

où $\mu \in \mathbb{R}$ est la moyenne (centre) et $\sigma^2 > 0$ la variance (dispersion).

13.2.2 La loi normale multivariée

Pour p variables observées simultanément, on généralise avec la loi normale multivariée $\mathcal{N}_p(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, de densité :

```
$$f(\boldsymbol{x}) = \frac{1}{(2\pi)^{p/2} |\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right), \quad \boldsymbol{x} \in \mathbb{R}^p$$
```

$$f(\boldsymbol{x}) = \frac{1}{(2\pi)^{p/2} |\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right), \quad \boldsymbol{x} \in \mathbb{R}^p$$

Les paramètres clés sont :

- $\boldsymbol{\mu} = (\mu_1, \dots, \mu_p)^\top$: le vecteur de moyennes (centre du nuage de points).
- $\boldsymbol{\Sigma}$: la matrice de covariance $p \times p$, symétrique définie positive, qui décrit la forme et l'orientation du nuage.

13.2.3 Le modèle de mélange gaussien

Notre jeu de données sera un mélange de $K = 3$ gaussiennes bivariées. La densité totale du mélange est :

```
$$f(\boldsymbol{x}) = \sum_{k=1}^K \pi_k \cdot \mathcal{N}_2(\boldsymbol{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k), \quad \sum_{k=1}^K \pi_k = 1, \pi_k > 0$$
```

$$f(\boldsymbol{x}) = \sum_{k=1}^K \pi_k \cdot \mathcal{N}_2(\boldsymbol{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k), \quad \sum_{k=1}^K \pi_k = 1, \pi_k > 0$$

```
data.frame(  
  Groupe      = c("Groupe 1", "Groupe 2", "Groupe 3"),  
  Proportion  = c("0.30", "0.40", "0.30"),  
  Moyenne     = c("(0, 0)", "(3, 5)", "(0, 6)"),  
  Covariance  = c("Sphérique I2", "Sphérique I2", "Elliptique Σell"),  
  Difficulte  = c("Facile", "Facile", "Difficile (ellipse inclinée)")  
) |>
```

```
setNames(c("Groupe", "Proportion  $\pi_k$ ", "Moyenne  $\mu_k$ ", "Covariance  $\Sigma_k$ ",
"Difficulté")) |>
  tt(caption = "Paramètres des trois composantes gaussiennes du mélange simulé.")
|>
  style_tt(j = 5, italic = TRUE)
```

Table 1

Groupe	Proportion π_k	Moyenne μ_k	Covariance Σ_k	Difficulté
Groupe 1	0.30	(0, 0)	Sphérique I_2	<i>Facile</i>
Groupe 2	0.40	(3, 5)	Sphérique I_2	<i>Facile</i>
Groupe 3	0.30	(0, 6)	Elliptique Σ_{ell}	<i>Difficile (ellipse inclinée)</i>

La matrice de covariance elliptique du Groupe 3 est :

```
$$\boldsymbol{\Sigma}_{ell} = \begin{pmatrix} 2 & 1.3 \\ 1.3 & 1 \end{pmatrix}$$$
```

$$\Sigma_{ell} = \begin{pmatrix} 2 & 1.3 \\ 1.3 & 1 \end{pmatrix}$$

Elle traduit une variance plus forte selon V_1 et une corrélation positive entre V_1 et V_2 (covariance = 1.3 > 0), ce qui crée l'ellipse inclinée à environ 45 degrés.

13.3 Simulation des variables continues

13.3.1 Vers une lecture plus nette : l'enveloppe convexe

Pour obtenir une frontière « physique » nette autour de chaque groupe, nous utilisons le concept d'**enveloppe convexe** (*Convex Hull*). Mathématiquement, l'enveloppe convexe d'un ensemble de points est le plus petit polygone convexe contenant tous ces points.

i Analogie pédagogique

Imaginez que chaque point de notre graphique soit un clou planté sur une planche. L'enveloppe convexe correspond à la forme que prendrait un élastique tendu que l'on relâcherait autour de tous les clous d'une même couleur. L'élastique s'appuierait uniquement sur les points les plus extérieurs, créant une frontière parfaite et sans « creux ».

13.3.2 Reproduction des couleurs originales de Flynt & Dean (2016)

```
set.seed(288)

pi_vec <- c(0.3, 0.4, 0.3)
mu <- list(c(0, 0), c(3, 5), c(0, 6))

sigma_sph <- diag(2)
sigma_ell <- matrix(c(2, 1.3, 1.3, 1), nrow = 2, byrow = TRUE)

cl_real <- sample(1:3, size = 600, replace = TRUE, prob = pi_vec)
```

```

X_data <- purrr::map_dfr(cl_real, function(i) {
  sigma_i <- if (i == 3L) sigma_ell else sigma_sph
  rmvnorm(n = 1, mean = mu[[i]], sigma = sigma_i) |> as_tibble()
}) |>
  set_names(c("V1", "V2"))

X_plot <- X_data |>
  mutate(Groupe = as.factor(cl_real)) |>
  filter(!is.na(V1) & !is.na(V2))

couleurs_article <- c("1" = "black", "2" = "red", "3" = "green3")

ggplot(X_plot, aes(x = V1, y = V2, color = Groupe)) +
  geom_point(alpha = 0.5, size = 1.2, shape = 18) +
  ggforce::geom_mark_hull(
    aes(fill = Groupe), alpha = 0.10, color = NA,
    concavity = 10000, na.rm = TRUE, expand = unit(2, "mm")
  ) +
  scale_color_manual(values = couleurs_article, name = "Groupe réel",
    labels = c("Groupe 1 (Black)", "Groupe 2 (Red)", "Groupe 3
(Green)")) +
  scale_fill_manual(values = couleurs_article) +
  labs(title = "Données simulées : séparation par enveloppes convexes",
    subtitle = "Reproduction des couleurs originales de Flynt & Dean (2016)",
    x = "V1", y = "V2") +
  theme_minimal() +
  theme(legend.position = "bottom") +
  guides(fill = "none")

```

Données simulées : séparation par enveloppes convexes

Reproduction des couleurs originales de Flynt & Dean (2016)

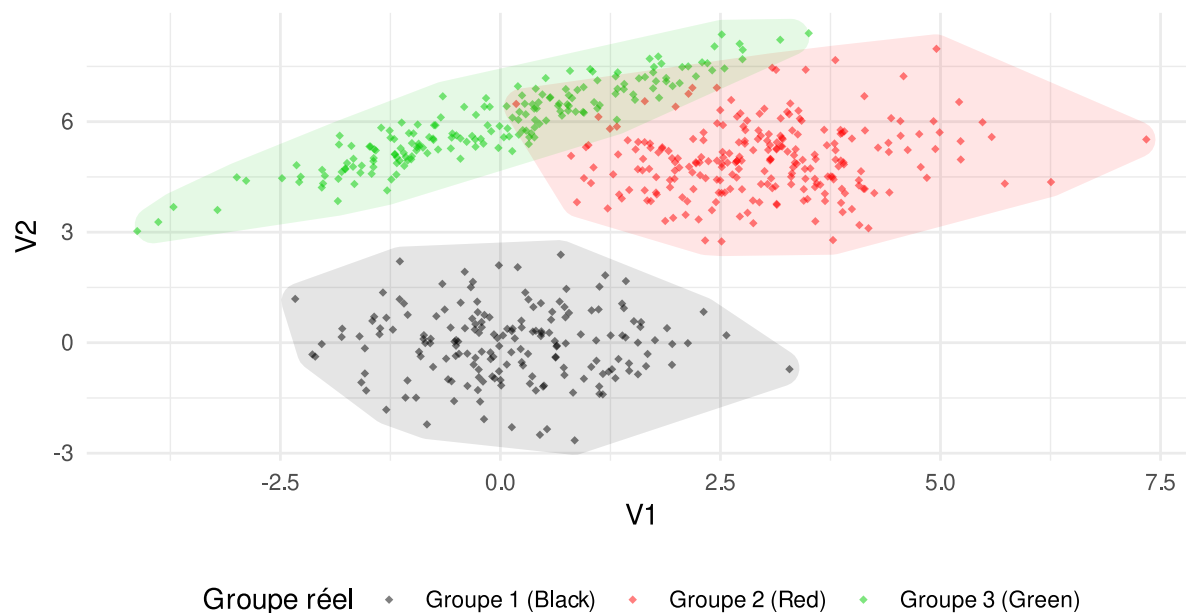


Figure 3

⚠ Point d'attention

Le Groupe 3 (en haut) forme une ellipse inclinée. Le K-means, qui suppose des clusters sphériques, aura du mal à le détecter correctement. Mclust, qui modélise explicitement les ellipses, le capturera bien.

14 Méthodes de partitionnement géométrique

14.1 K-means : la méthode des barycentres

14.1.1 Principe et intuition

Le K-means est l'algorithme de clustering le plus célèbre et le plus utilisé au monde [5], [9]. Son principe :

Chaque individu appartient au groupe dont le centre (la moyenne) lui est le plus proche.

i Analogie pédagogique

Imaginez 3 personnes dans une salle bondée, chacune criant : « Venez vers moi ! » Chaque individu rejoint la plus proche. Puis chaque personne se déplace au centre géographique de son groupe. On recommence jusqu'à stabilisation. C'est exactement l'algorithme K-means.

14.1.2 Formalisation mathématique

K-means minimise la **somme des distances au carré intra-cluster** (WSS, *Within-Cluster Sum of Squares*) :

$$\mathrm{WSS} = \sum_{k=1}^K \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

14.1.3 Choisir K : la méthode du coude

```
elbow_data <- tibble(k = 1:9) |>
  mutate(
    model = purrr::map(k, ~ kmeans(X_data, centers = .x, nstart = 50)),
    wss    = purrr::map_dbl(model, ~ .x$tot.withinss)
  )

ggplot(elbow_data, aes(x = k, y = wss)) +
  geom_line(color = "gray70", linewidth = 0.9) +
  geom_point(aes(color = (k == 3)), size = 3.5) +
  scale_color_manual(values = c("black", "#B40000")) +
  scale_x_continuous(breaks = 1:9) +
  labs(title = "Inertie intra-classe selon le nombre de clusters K",
       x     = "Nombre de clusters K",
       y     = "WSS (Within-Cluster Sum of Squares)") +
  guides(color = "none")
```

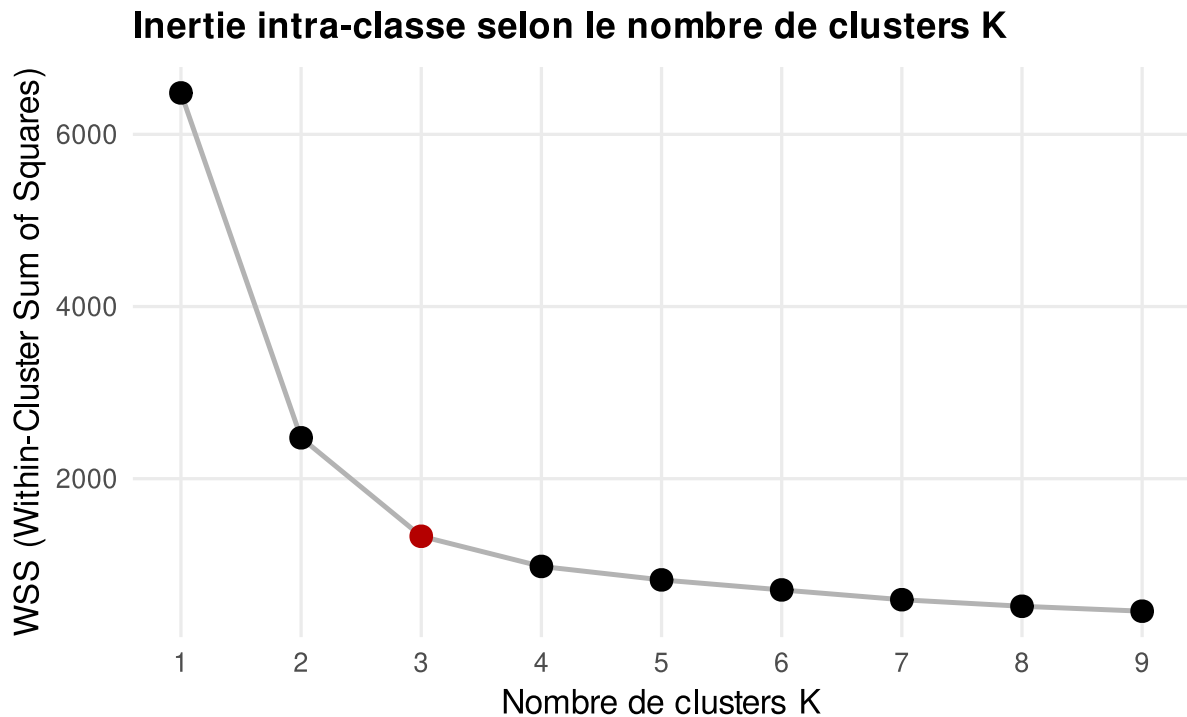


Figure 4

⚠ Limites du K-means

- (1) il suppose des clusters sphériques, inadapté aux formes elliptiques ;
- (2) il est sensible aux valeurs aberrantes ;
- (3) il n'exprime aucune incertitude : chaque individu est assigné définitivement à un seul groupe ;
- (4) il ne fonctionne que sur des variables continues.

14.1.4 Six variations pour générer la courbe WSS

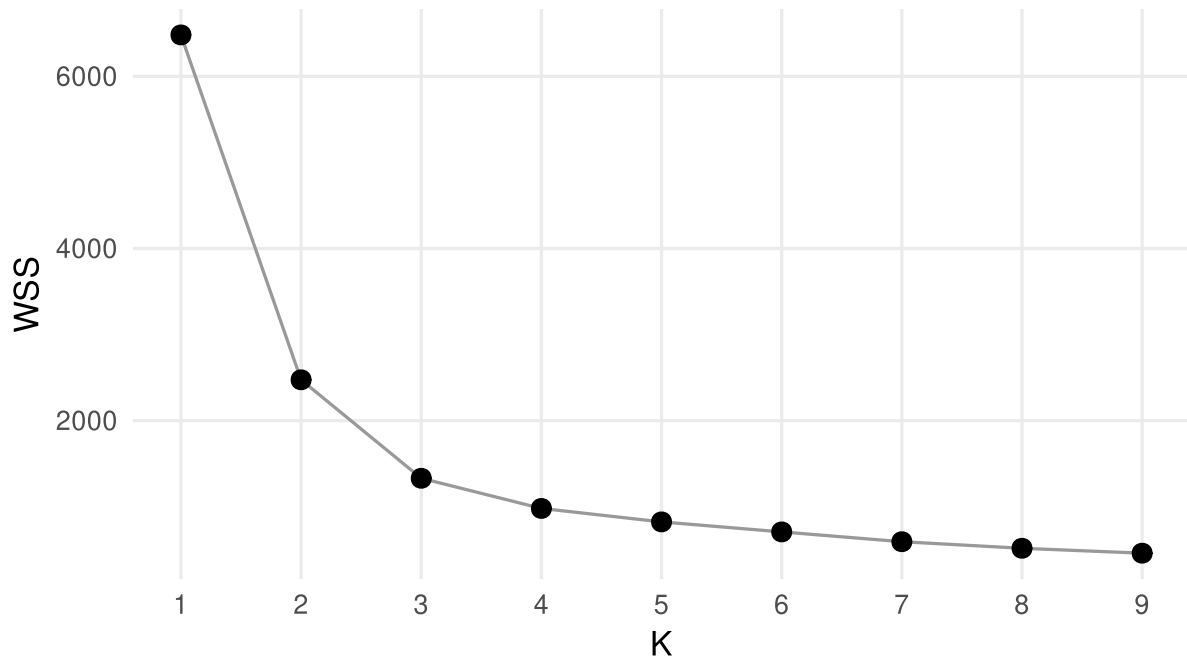
Variation 1 – Méthode manuelle

```
km1 <- kmeans(X_data, 1, nstart = 50); km2 <- kmeans(X_data, 2, nstart = 50)
km3 <- kmeans(X_data, 3, nstart = 50); km4 <- kmeans(X_data, 4, nstart = 50)
km5 <- kmeans(X_data, 5, nstart = 50); km6 <- kmeans(X_data, 6, nstart = 50)
km7 <- kmeans(X_data, 7, nstart = 50); km8 <- kmeans(X_data, 8, nstart = 50)
km9 <- kmeans(X_data, 9, nstart = 50)

wss_man <- c(km1$tot.withinss, km2$tot.withinss, km3$tot.withinss,
             km4$tot.withinss, km5$tot.withinss, km6$tot.withinss,
             km7$tot.withinss, km8$tot.withinss, km9$tot.withinss)

ggplot(tibble(k = 1:9, wss = wss_man), aes(k, wss)) +
  geom_line(color = "gray60") + geom_point(size = 3) +
  scale_x_continuous(breaks = 1:9) +
  labs(title = "Variation 1 : Méthode manuelle", x = "K", y = "WSS")
```


Variation 1 : Méthode manuelle

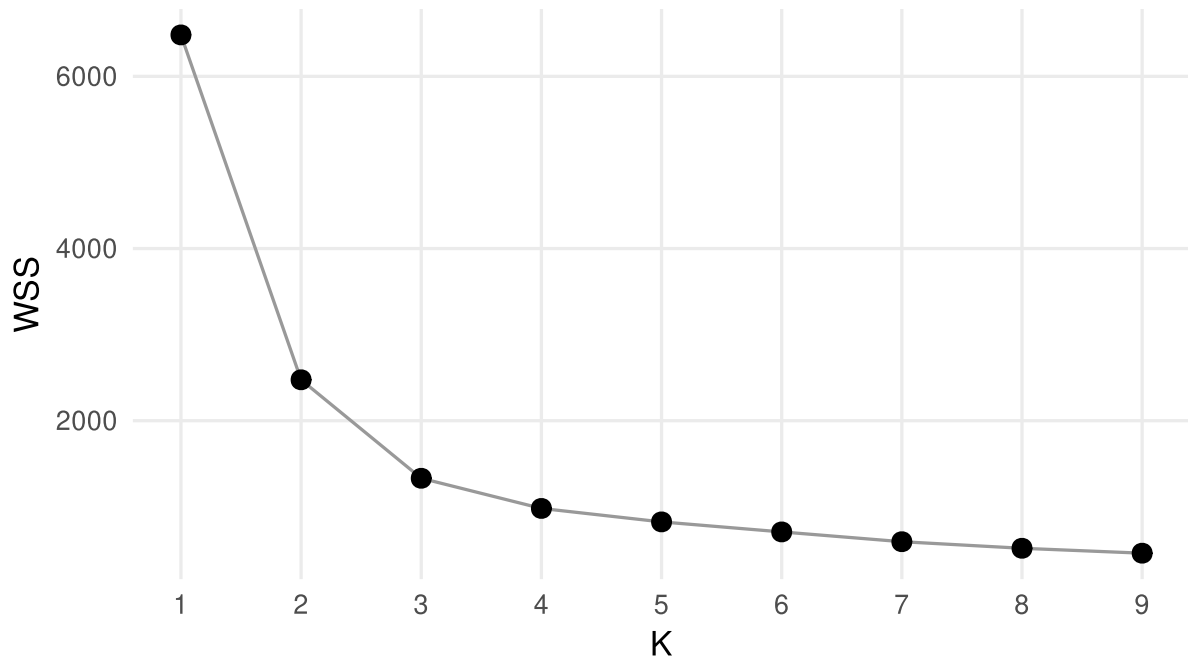


Variation 2 – Boucle for

```
wss_for <- numeric(9)
for (i in 1:9) {
  wss_for[i] <- kmeans(X_data, centers = i, nstart = 50)$tot.withinss
}

ggplot(tibble(k = 1:9, wss = wss_for), aes(k, wss)) +
  geom_line(color = "gray60") + geom_point(size = 3) +
  scale_x_continuous(breaks = 1:9) +
  labs(title = "Variation 2 : Boucle For", x = "K", y = "WSS")
```

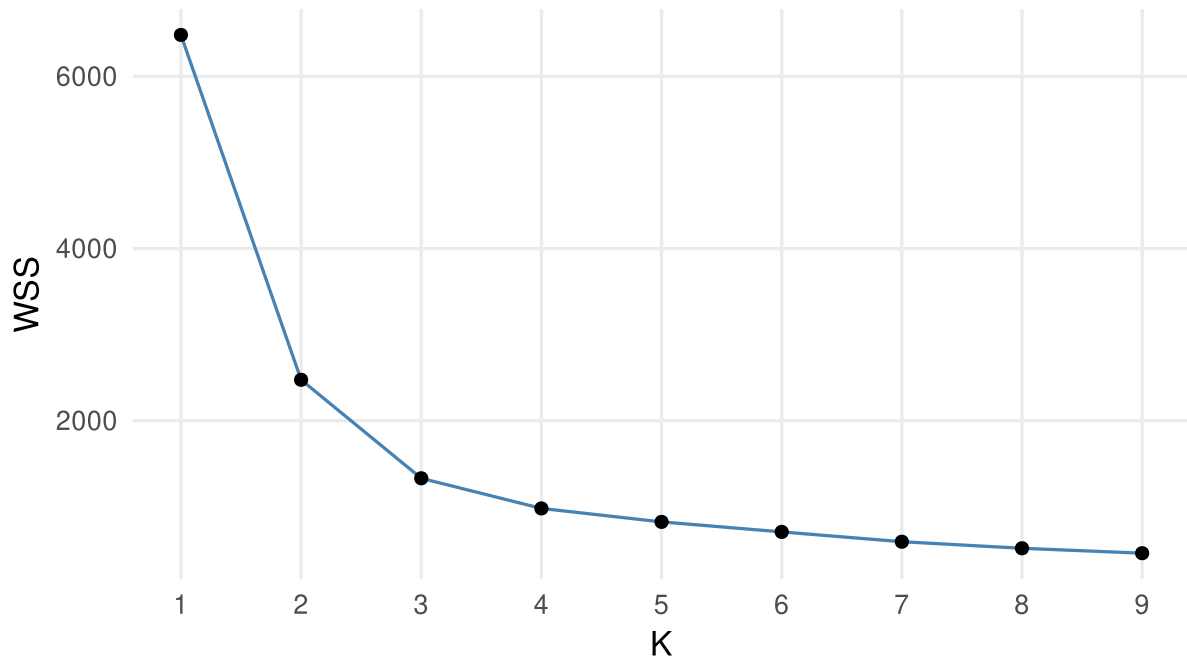
Variation 2 : Boucle For



Variation 3 — sapply

```
wss_sapply <- sapply(1:9, function(k) {  
  kmeans(X_data, centers = k, nstart = 50)$tot.withinss  
})  
  
ggplot(tibble(k = 1:9, wss = wss_sapply), aes(k, wss)) +  
  geom_line(color = "steelblue") + geom_point() +  
  scale_x_continuous(breaks = 1:9) +  
  labs(title = "Variation 3 : sapply (Vectorisation)", x = "K", y = "WSS")
```

Variation 3 : supply (Vectorisation)

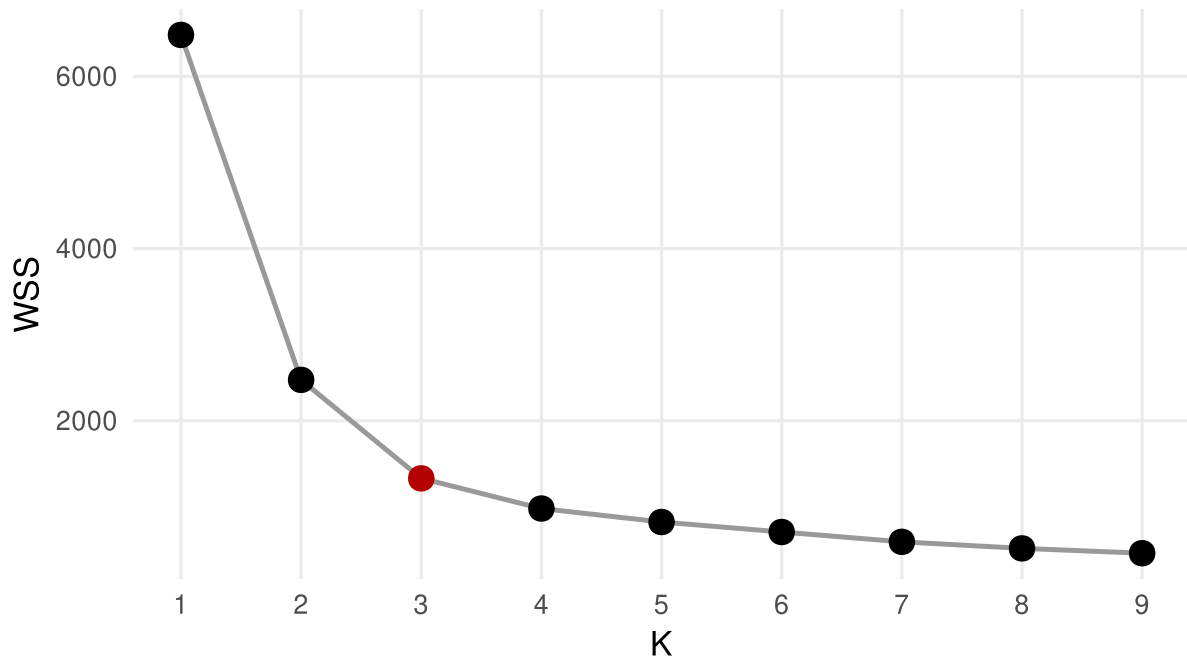


Variation 4 — purrr

```
elbow_purrr <- tibble(k = 1:9) |>
  mutate(
    model = purrr::map(k, ~ kmeans(X_data, centers = .x, nstart = 50)),
    wss = purrr::map_dbl(model, ~ .x$tot.withinss)
  )

ggplot(elbow_purrr, aes(k, wss)) +
  geom_line(linewidth = 0.9, color = "gray60") +
  geom_point(aes(color = (k == 3)), size = 4) +
  scale_color_manual(values = c("black", "#B40000"), guide = "none") +
  scale_x_continuous(breaks = 1:9) +
  labs(title = "Variation 4 : purrr (Programmation fonctionnelle)", x = "K", y =
"WSS")
```

Variation 4 : purrr (Programmation fonctionnelle)

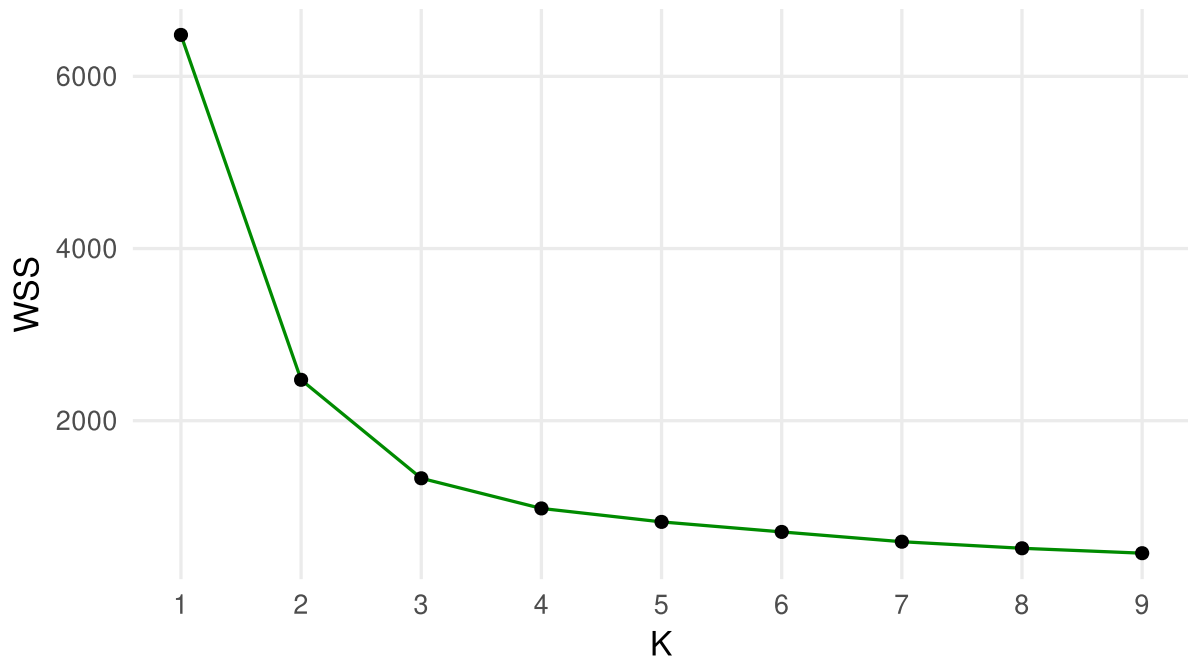


Variation 5 – broom

```
elbow_broom <- tibble(k = 1:9) |>
  mutate(
    model = purrr::map(k, ~ kmeans(X_data, centers = .x, nstart = 50)),
    stats = purrr::map(model, broom::glance)
  ) |>
  tidyr::unnest(stats)

ggplot(elbow_broom, aes(k, tot.withinss)) +
  geom_line(color = "green4") + geom_point() +
  scale_x_continuous(breaks = 1:9) +
  labs(title = "Variation 5 : broom (métriques standardisées)", y = "WSS", x =
"K")
```

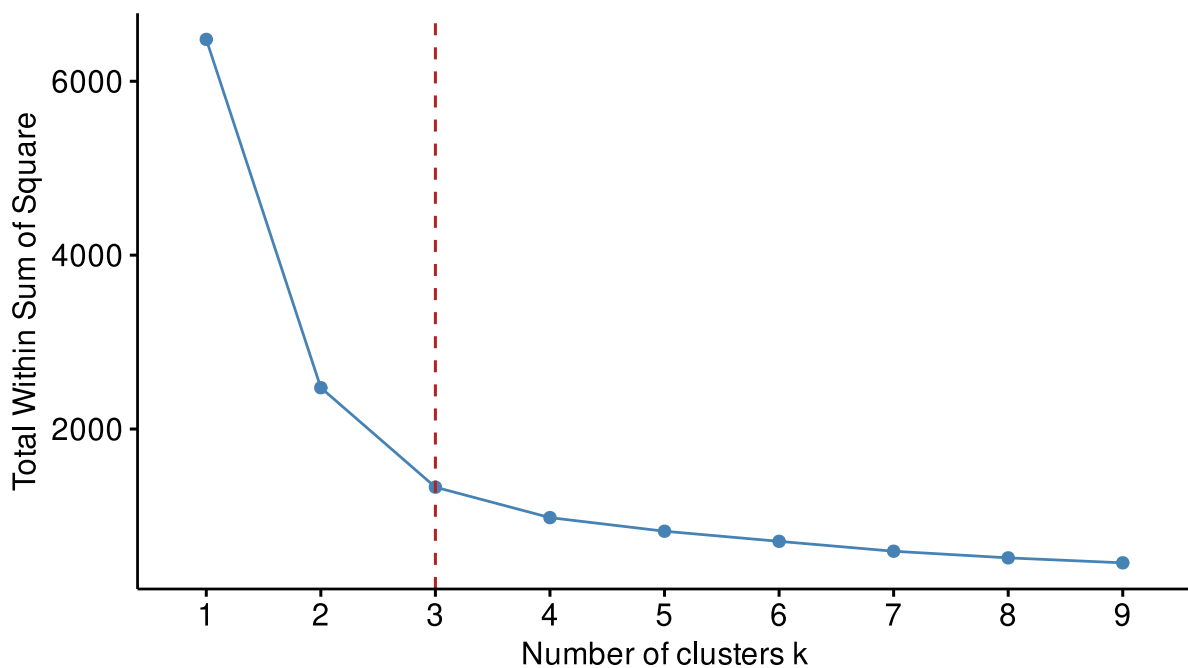
Variation 5 : broom (métriques standardisées)



Variation 6 – factoextra (clé en main)

```
fviz_nbclust(X_data, kmeans, method = "wss", k.max = 9, nstart = 50) +  
  geom_vline(xintercept = 3, linetype = 2, color = "firebrick") +  
  labs(title = "Variation 6 : factoextra (Clé en main)")
```

Variation 6 : factoextra (Clé en main)



💡 À retenir

Les six courbes WSS produites sont **statistiquement identiques** : toutes indiquent un coude en $k = 3$, confirmant exactement la structure simulée par Flynt & Dean. Le choix de la variation est donc un choix de **style de code** et de contexte d'usage.

```
data.frame(  
  Methode = c("Manuelle", "Boucle For", "sapply", "purrr", "broom",  
    "factoextra"),  
  Avantage = c("Transparence totale", "Logique universelle", "Concis (Base R)",  
    "Traçabilité complète", "Métriques multiples", "Rapidité,  
    esthétique"),  
  Inconvenient = c("Très verbeux", "Plus de lignes", "Modèle perdu",  
    "Syntaxe complexe", "Package sup.", "\"Boîte noire\""),  
  Usage = c("Apprentissage", "Débutants", "Scripts légers",  
    "Pipelines DS", "Rapports stats", "Exploration")  
) |>  
  setNames(c("Méthode", "Avantage principal", "Inconvénient", "Usage idéal")) |>  
  tt(caption = "Tableau de synthèse des six variations d'implémentation de la  
  méthode du coude.")
```

Table 2

Méthode	Avantage principal	Inconvénient	Usage idéal
Manuelle	Transparence totale	Très verbeux	Apprentissage
Boucle For	Logique universelle	Plus de lignes	Débutants
sapply	Concis (Base R)	Modèle perdu	Scripts légers
purrr	Traçabilité complète	Syntaxe complexe	Pipelines DS
broom	Métriques multiples	Package sup.	Rapports stats
factoextra	Rapidité, esthétique	« Boîte noire »	Exploration

14.2 Superposition K-means / vérité terrain

```
set.seed(123)  
km_res <- kmeans(X_data, centers = 3, nstart = 25)  
  
X_eval <- X_data |>  
  mutate(  
    Vrai_Groupe = as.factor(cl_real),  
    Cluster_KM = as.factor(km_res$cluster)  
  )  
  
ggplot(X_eval, aes(x = V1, y = V2)) +  
  geom_point(aes(color = Vrai_Groupe), alpha = 0.6, size = 1.2, shape = 18) +  
  ggforce::geom_mark_hull(
```

```

aes(fill = Cluster_KM, group = Cluster_KM),
alpha = 0.10, color = "gray40", linetype = "solid",
concavity = 10000, radius = unit(0, "mm"),
expand = unit(0, "mm"), na.rm = TRUE
) +
scale_color_manual(values = couleurs_article, name = "Vérité Terrain") +
scale_fill_manual(values = couleurs_article) +
labs(title = "Évaluation K-means : Enveloppes Convexes Strictes",
      subtitle = "Polygones reliant les points extrêmes de chaque cluster",
      x = "V1", y = "V2") +
theme_minimal() +
theme(legend.position = "bottom") +
guides(fill = "none")

```

Évaluation K-means : Enveloppes Convexes Strictes

Polygones reliant les points extrêmes de chaque cluster

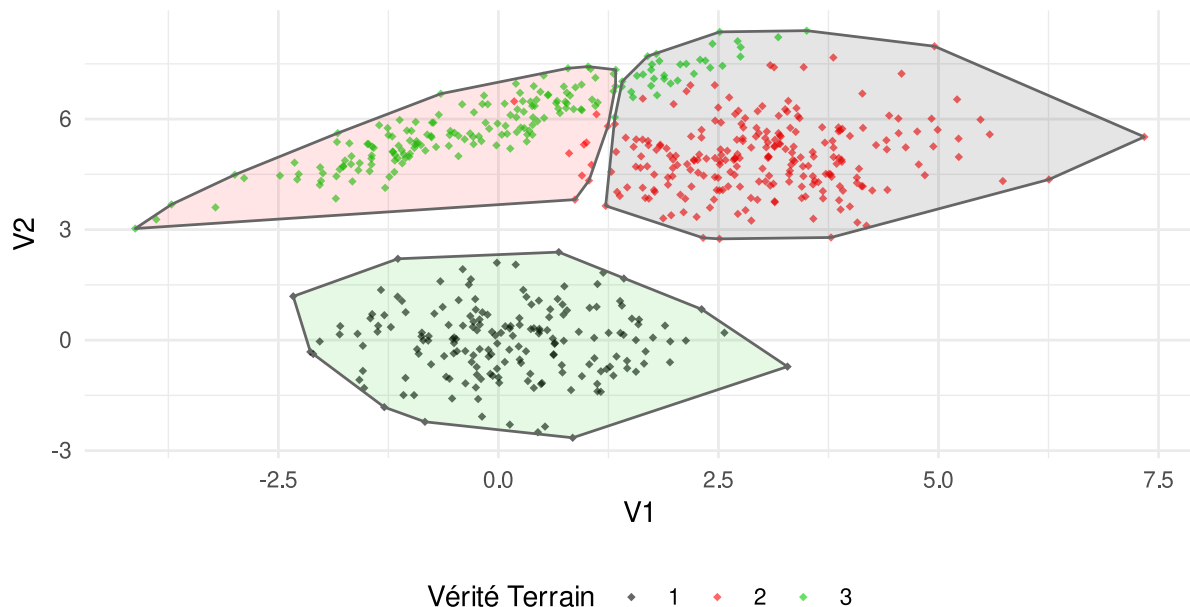


Figure 5

14.2.1 Performance du K-means (K=3)

```

km3      <- kmeans(X_data, 3, nstart = 50)
tab_km   <- table(cl_real, km3$cluster)
cat("Table de contingence K-means vs vérité terrain :\n")

```

```
#> Table de contingence K-means vs vérité terrain :
```

```
print(tab_km)
```

```
#>
#> cl_real  1   2   3
#>      1   0   0 177
#>      2 227  10   0
#>      3  33 153   0
```

```
reussite_points <- sum(apply(tab_km, 1, max))
total_points    <- sum(tab_km)
taux_reussite   <- (reussite_points / total_points) * 100
taux_erreur     <- 100 - taux_reussite

cat("\nTaux de réussite :", round(taux_reussite, 2), "%")
```

```
#>
#> Taux de réussite : 92.83 %
```

```
cat("\nTaux d'erreur   :", round(taux_erreur, 2), "%")
```

```
#>
#> Taux d'erreur     : 7.17 %
```

```
data.frame(
  Indicateur = c("Points bien classés", "Taux de réussite", "Taux d'erreur"),
  Resultat   = c(paste0(reussite_points, " / ", total_points),
                 paste0(round(taux_reussite, 2), " %"),
                 paste0(round(taux_erreur, 2), " %"))
) |>
  setNames(c("Indicateur de performance", "Résultat")) |>
  tt(caption = "Analyse de la précision du K-means (K=3) sur les données Flynt &
Dean.")
```

Table 3

Indicateur de performance	Résultat
Points bien classés	557 / 600
Taux de réussite	92.83 %
Taux d'erreur	7.17 %

⚠ En conclusion

Si le Groupe 1 valide l'efficacité de l'approche dans des conditions simples, les Groupes 2 et 3 démontrent que la propriété visuelle d'un clustering ne garantit pas sa justesse statistique. Le K-means impose sa propre géométrie (sphérique) aux données au lieu de s'adapter à la leur (elliptique).

14.3 Classification hiérarchique ascendante (CHA)

14.3.1 Principe

La **classification hiérarchique ascendante** (CHA) construit un **dendrogramme** — un arbre de fusion représentant quels individus ou groupes ont été réunis, et à quelle distance [7]. Elle ne requiert pas de spécifier K à l'avance.

La liaison moyenne (UPGMA) définit la distance entre deux groupes A et B comme la moyenne de toutes les distances inter-individuelles :

$$d(A, B) = \frac{1}{|A| \cdot |B|} \sum_{i \in A} \sum_{j \in B} d(x_i, x_j)$$

$$d(A, B) = \frac{1}{|A| \cdot |B|} \sum_{i \in A} \sum_{j \in B} d(x_i, x_j)$$

```
d_mat <- dist(X_data, method = "euclidean")
h_avg <- hclust(d_mat, method = "average")

ggdendrogram(h_avg, rotate = FALSE, size = 0.15) +
  labs(
    title = "Dendrogramme de classification hiérarchique",
    subtitle = "Liaison moyenne (UPGMA) --- 600 individus"
  ) +
  theme(axis.text.x = element_blank(),
        axis.ticks.x = element_blank())
```

Dendrogramme de classification hiérarchique

Liaison moyenne (UPGMA) --- 600 individus

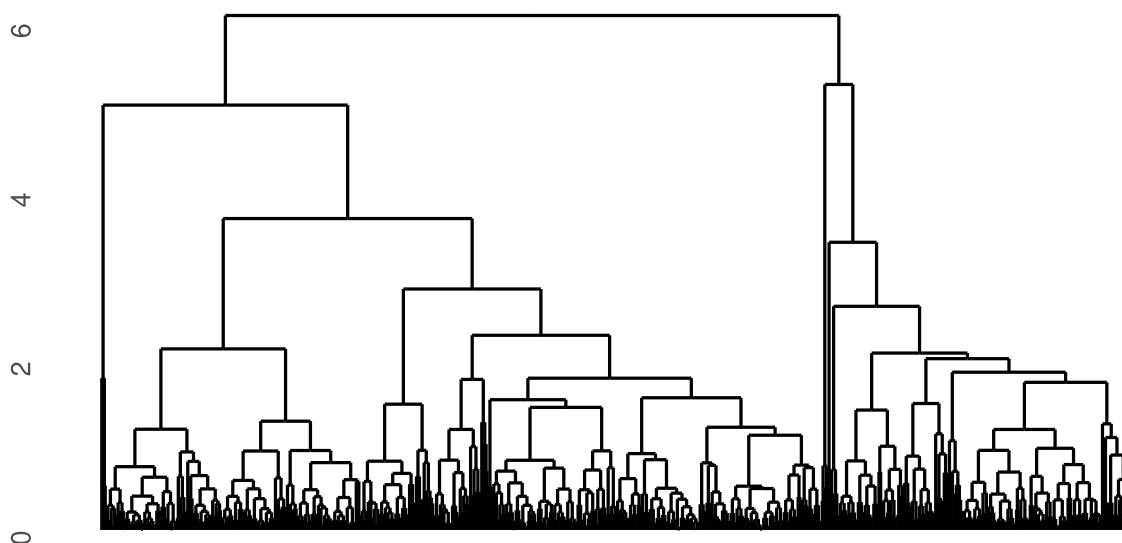


Figure 6. – Dendrogramme de classification hiérarchique (liaison moyenne, 600 individus). La hauteur d'une jonction représente la distance de fusion. Pour $K = 3$, on coupe au niveau de la 3e plus grande fusion.

14.3.2 La méthode hiérarchique CAH sur les données simulées

Dans l'article de Flynt & Dean (2016), la méthode du lien moyen peut s'avérer encore moins performante que le K-means sur ces structures, entraînant un taux d'erreur avoisinant les 30 %. La raison fondamentale : la CAH fusionne des sous-groupes de façon **irréversible** (*greedy*), sans pouvoir réassigner un point mal placé.

```
res.hc <- hclust(dist(X_data), method = "average")

fviz_dend(res.hc,
  k          = 3,
  cex        = 0.15,
  lwd        = 0.30,
  rect       = TRUE,
  rect_fill  = TRUE,
  rect_lty   = 1,
  palette    = c("black", "red", "green3"),
  main       = "Dendrogramme (Average Linkage)",
  xlab       = "Observations",
  ylab       = "Distance Euclidienne",
  ggtheme    = theme_minimal() +
  scale_linewidth(range = c(0.30, 0.30)) +
  theme(
    axis.text.x    = element_blank(),
    legend.position = "none"
  )
)
```

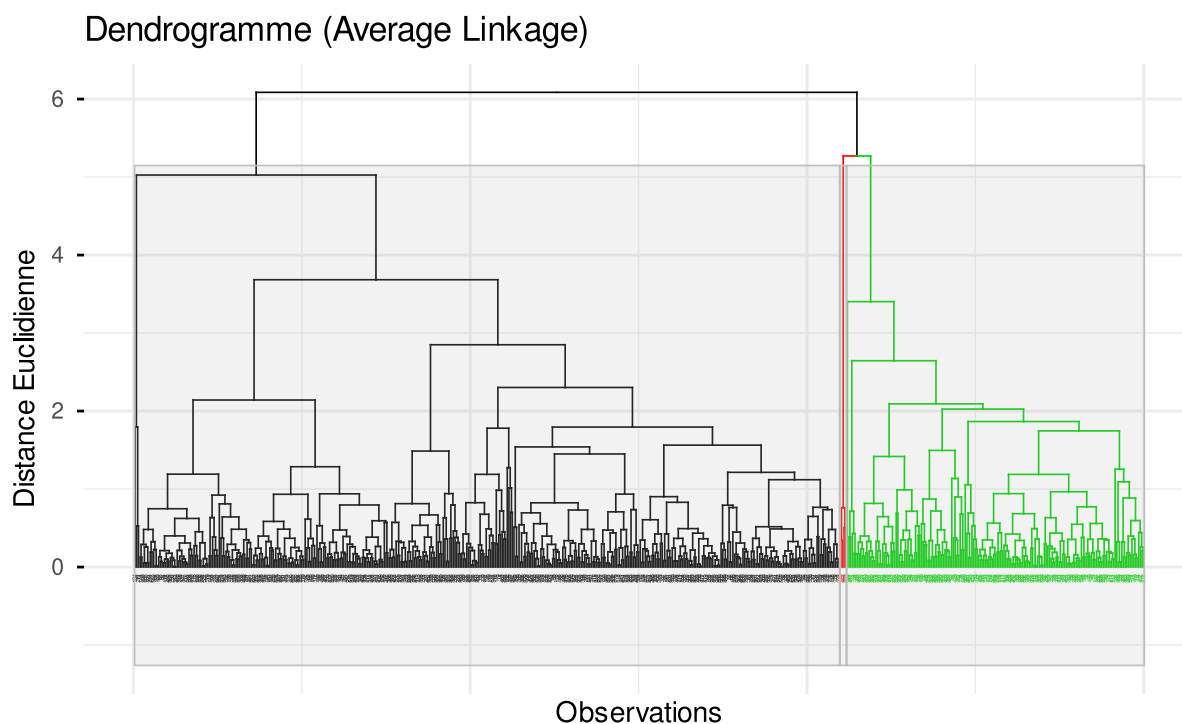


Figure 7. – Dendrogramme (lien moyen, 600 individus) avec découpage en 3 clusters selon les couleurs de l'article.

14.3.3 La version circulaire

```
library(patchwork)
library(grid)

# 1. Construction du plot
p_circ <- fviz_dend(res.hc,
                    k           = 3,
                    type        = "circular",
                    cex         = 0.15,
                    rect        = TRUE,
                    rect_fill   = TRUE,
                    palette     = c("black", "red", "green3"),
                    main        = "Dendrogramme Circulaire (Average Linkage)",
                    ggtheme     = theme_void())

# 2. Patch linewidth sur tous les segments
for (i in seq_along(p_circ$layers)) {
  geom_class <- class(p_circ$layers[[i]]$geom)
  if (any(geom_class %in% c("GeomSegment", "GeomCurve", "GeomPath"))) {
    p_circ$layers[[i]]$aes_params$linewidth <- 0.30
    p_circ$layers[[i]]$aes_params$size      <- 0.30
  }
}

p_circ <- p_circ +
  scale_linewidth(range = c(0.30, 0.30)) +
```

```

theme_void() +
theme(legend.position = "none")

# 3. Suppression des labels radiaux
gb <- ggplot_build(p_circ)
gb$layout$panel_params[[1]]$r.labels <- character(0)
gb$layout$panel_params[[1]]$r.major <- numeric(0)
gb$layout$panel_params[[1]]$r.minor <- numeric(0)
gt <- ggplot_gtable(gb)

# 4. Rendu compatible Quarto
wrap_elements(gt)

```

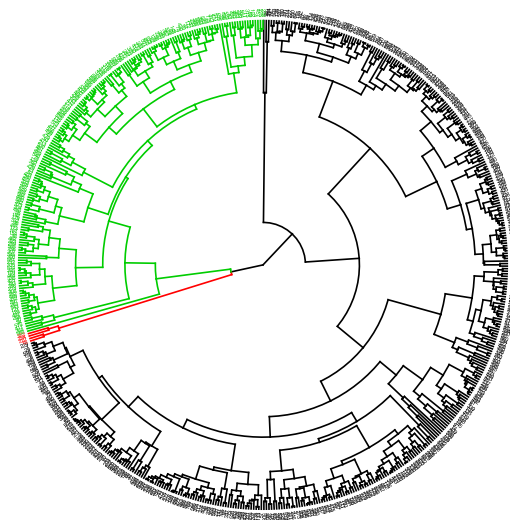


Figure 8. – Dendrogramme circulaire (lien moyen, 600 individus) avec découpage en 3 clusters.

14.3.4 Le calcul (30 % d'erreur)

Pour retrouver les chiffres de l'article, il faut regarder si les points de **différentes couleurs** sont mélangés dans le même cluster :

```

hRed <- hclust(dist(X_data[1:2]), method = "average")
H2cut <- cutree(hRed, k = 3)
table(cl_real, H2cut)

```

```

#>      H2cut
#> cl_real  1  2  3
#>      1   0 177  0

```

#>	2 237	0	0
#>	3 182	0	4

Taux de réussite : $(237 + 177 + 4)/600 = 69,7\%$ — Taux d'erreur : $182/600 = 30,3\%$

⚠ Constat de Flynt & Dean

Alors que le K-means affiche environ 7–10 % d'erreur, la méthode hiérarchique dépasse ici les **30 % d'erreur**. Les deux groupes sphériques, trop proches, sont fusionnés en un « super-cluster » artificiel par le lien moyen, tandis que le groupe vert (elliptique) se retrouve fragmenté.

💡 À retenir

La méthode hiérarchique est utile pour explorer la structure des données, mais pour un partitionnement robuste sur ces données simulées, le K-means (et a fortiori Mclust) reste largement supérieur.

15 Méthodes basées sur les modèles probabilistes

15.1 Modèles de mélanges gaussiens : Mclust

15.1.1 Du déterministe au probabiliste

Avec K-means et la CHA, chaque individu est assigné à exactement un groupe, sans la moindre nuance. Les **Modèles de Mélanges Gaussiens** (GMM) apportent une vision plus honnête : chaque individu appartient à chaque groupe avec une certaine probabilité. Formellement :

$$f(\mathbf{x}) = \sum_{k=1}^K \pi_k \cdot \mathcal{N}_p(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

i Analogie pédagogique

Le K-means dit : « Tu es un client premium, point. » Le GMM dit : « Tu es probablement un client premium à 75 %, standard à 20 % et VIP à 5 %. » C'est nettement plus nuancé et réaliste.

15.1.2 L'algorithme EM

Les paramètres $\{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1}^K$ sont estimés par l'algorithme EM [10] :

Étape E (Expectation) — calcul des responsabilités r_{ik} :

```


$$r_{ik} = \frac{\pi_k \cdot \mathcal{N}_p(x_i | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \cdot \mathcal{N}_p(x_i | \mu_j, \Sigma_j)}$$


```

$$r_{ik} = \frac{\pi_k \cdot \mathcal{N}_p(x_i | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \cdot \mathcal{N}_p(x_i | \mu_j, \Sigma_j)}$$

Étape M (Maximization) – mise à jour des paramètres :

```


$$\hat{\pi}_k = \frac{1}{n} \sum_i r_{ik}, \quad \hat{\mu}_k = \frac{\sum_i r_{ik} x_i}{\sum_i r_{ik}}, \quad \hat{\Sigma}_k = \frac{\sum_i r_{ik} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^T}{\sum_i r_{ik}}$$


```

$$\hat{\pi}_k = \frac{1}{n} \sum_i r_{ik}, \quad \hat{\mu}_k = \frac{\sum_i r_{ik} x_i}{\sum_i r_{ik}}, \quad \hat{\Sigma}_k = \frac{\sum_i r_{ik} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^T}{\sum_i r_{ik}}$$

15.1.3 Sélection automatique via le BIC

Mclust [11], [13] sélectionne automatiquement K et la structure de covariance en maximisant le **Critère d'Information Bayésien (BIC)** [14] :

```


$$\text{BIC} = 2 \ln \hat{L} - d \ln n$$


```

$$\text{BIC} = 2 \ln \hat{L} - d \ln n$$

```

mc_model_v2 <- Mclust(X_data[, 1:2], G = 3)
tab_mc_v2    <- table(Vérité = cl_real, Mclust = mc_model_v2$classification)
print(tab_mc_v2)

```

```

#>      Mclust
#> Vérité   1   2   3
#>      1    0 177   0
#>      2 229   0   8
#>      3   0   0 186

```

```

reussite_mc      <- sum(apply(tab_mc_v2, 1, max))
taux_reussite_mc <- (reussite_mc / nrow(X_data)) * 100
taux_erreur_mc   <- 100 - taux_reussite_mc

cat("Taux de réussite Mclust :", round(taux_reussite_mc, 2), "%")

```

```

#> Taux de réussite Mclust : 98.67 %

```

```

cat("\nTaux d'erreur Mclust      :", round(taux_erreur_mc, 2), "%")

```

```
#>
#> Taux d'erreur Mclust      : 1.33 %
```

```
data.frame(
  Indicateur = c("Points bien capturés", "Taux de réussite (Capture)", "Taux
d'erreur"),
  Resultat   = c(paste0(reussite_mc, " / ", nrow(X_data)),
                 paste0(round(taux_reussite_mc, 2), " %"),
                 paste0(round(taux_erreur_mc, 2), " %"))
) |>
  setNames(c("Indicateur", "Résultat")) |>
  tt(caption = "Performance du modèle Mclust (G=3) sur les données Flynt & Dean.")
|>
  style_tt(i = 2, bold = TRUE, color = "white", background = "#28a745")
```

Table 4

Indicateur	Résultat
Points bien capturés	592 / 600
Taux de réussite (Capture)	98.67 %
Taux d'erreur	1.33 %

```
mc_res_v2 <- Mclust(X_data[, 1:2], G = 3)

X_mclust_v2 <- X_data |>
  mutate(
    Verite      = as.factor(cl_real),
    Cluster_Mclust = as.factor(mc_res_v2$classification)
  )

bleu_v2 <- rgb(0, 70, 127, maxColorValue = 255)

ggplot(X_mclust_v2, aes(x = V1, y = V2)) +
  geom_point(aes(color = Verite), alpha = 0.6, size = 1.2, shape = 18) +
  ggforce::geom_mark_hull(
    aes(fill = Cluster_Mclust, group = Cluster_Mclust,
        label = paste("Cluster", Cluster_Mclust)),
    alpha = 0.10, color = bleu_v2, linetype = "solid",
    concavity = 10000, radius = unit(0, "mm"), expand = unit(0, "mm"),
    label.buffer = unit(5, "mm"), label.fontsize = 9,
    label.fontface = "bold", na.rm = TRUE
  ) +
  scale_color_manual(values = couleurs_article, name = "Vérité Terrain") +
  scale_fill_manual(values = couleurs_article) +
  labs(title = "Mclust : Identification des structures gaussiennes",
        subtitle = "Les étiquettes désignent les populations identifiées par
```

```
l'algorithme",
  x = "Variable V1", y = "Variable V2") +
theme_minimal() +
theme(legend.position = "bottom", plot.title = element_text(face = "bold")) +
guides(fill = "none")
```

Mclust : Identification des structures gaussiennes

Les étiquettes désignent les populations identifiées par l'algorithme

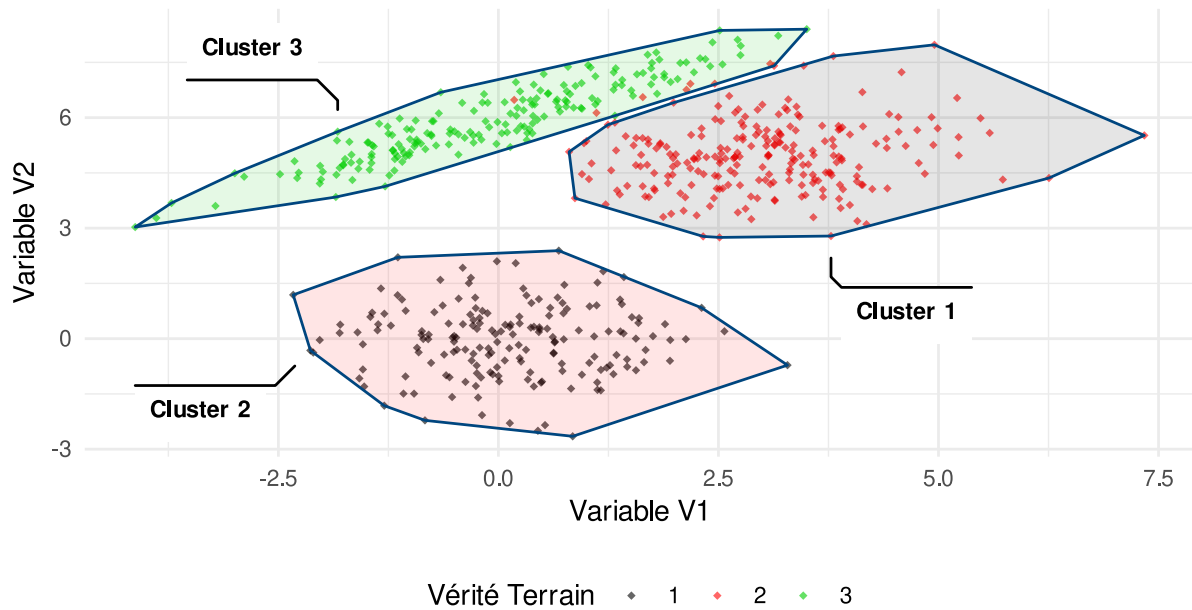


Figure 9

```
mc_full_fviz <- Mclust(X_data[, 1:2], G = 1:9)
fviz_mclust_bic(mc_full_fviz, palette = "jco", legend = "right",
  ggtheme = theme_minimal())
```

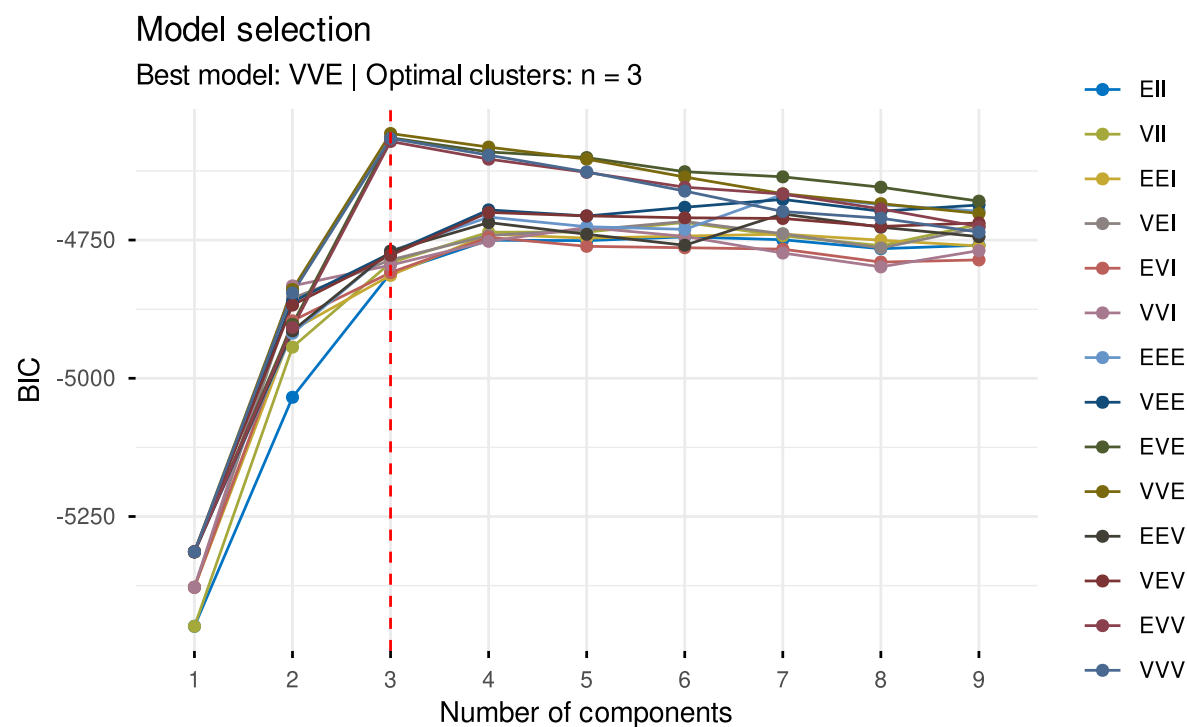



Figure 10

```
fviz_mclust(mc_res_v2, what = "classification",
  palette = c("black", "red", "green3"),
  ggtheme = theme_minimal(),
  main = "Classification Mclust (G=3)")
```

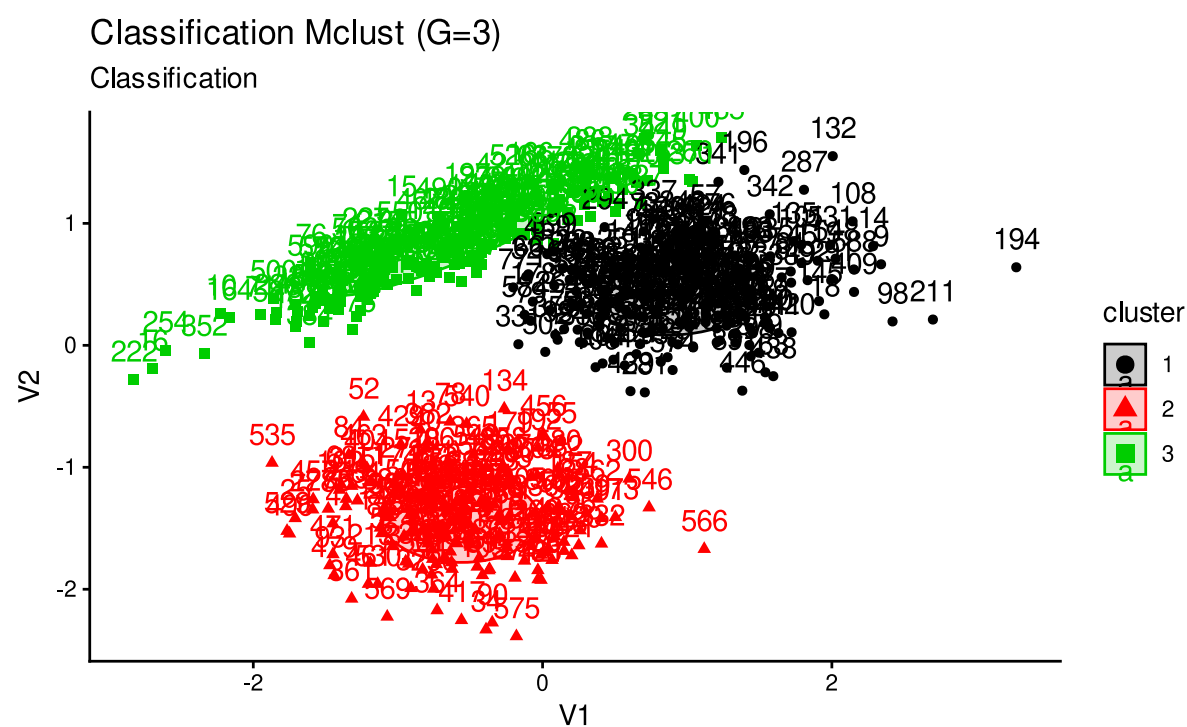


Figure 11

```
ggplot(X_mclust_v2, aes(x = V1, y = V2, color = Cluster_Mclust)) +
  geom_point(alpha = 0.5, size = 1.5, shape = 16) +
  stat_ellipse(aes(fill = Cluster_Mclust), geom = "polygon", alpha = 0.1, level =
0.95) +
  scale_color_manual(values = c("black", "red", "green3")) +
  scale_fill_manual(values = c("black", "red", "green3")) +
  labs(title = "Structure des populations identifiées",
  subtitle = "Les ellipses de confiance illustrent la modélisation
gaussienne")
```

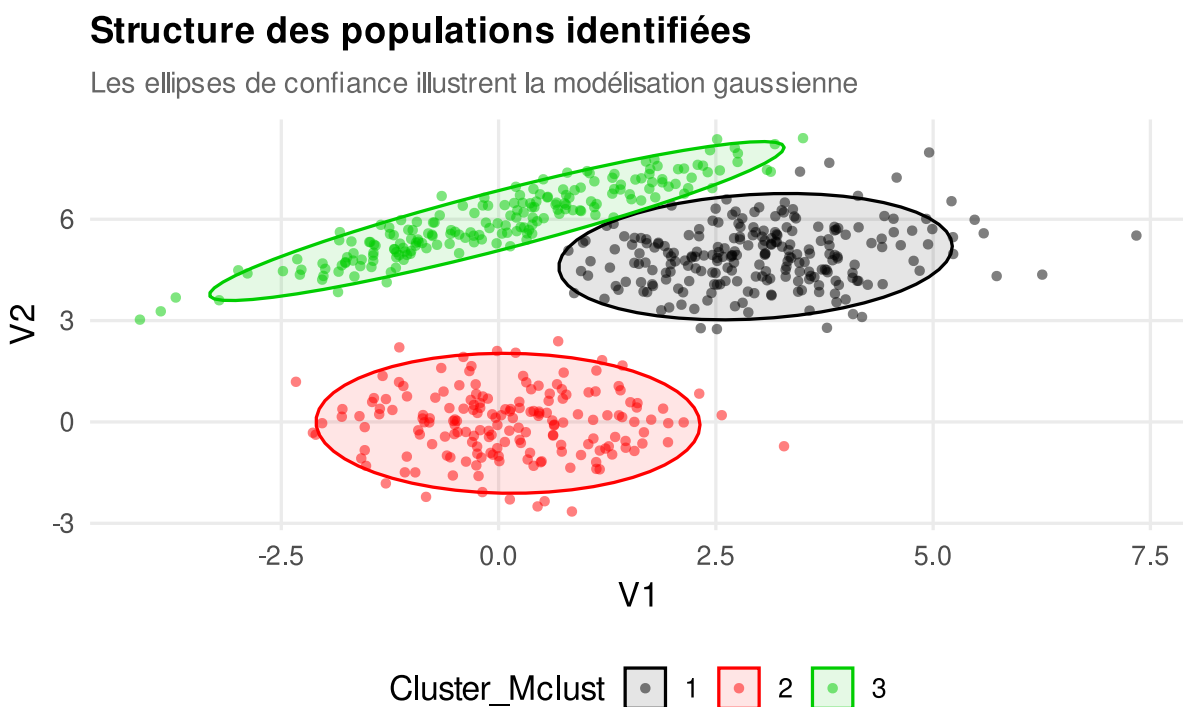


Figure 12

```
X_mclust_v2$Uncertainty <- mc_res_v2$uncertainty

ggplot(X_mclust_v2, aes(x = V1, y = V2, color = Uncertainty)) +
  geom_point(size = 2) +
  scale_color_gradient(low = "gray80", high = "firebrick1") +
  labs(title = "Zones de doute de l'algorithme Mclust",
  subtitle = "Les points rouges marquent les frontières de décision
ambiguës",
  color = "Incertitude")
```

Les points rouges marquent les frontières de décision ambiguës


$$T_{\text{réussite}} = \frac{177 + 229 + 186}{600} = \frac{592}{600} \approx 0{,}9867 \quad \rightarrow \quad 98{,}67\%$$

💡 Le triomphe de Mclust

16 Comparaison et synthèse des méthodes

ESD113 — Probabilités et statistique avec R

```

cl_cah <- cutree(res_hc_cmp, k = 3)
ari_cah <- round(mclust::adjustedRandIndex(cl_cah, cl_real), 3)

mc_cmp <- Mclust(X_data[, 1:2], G = 3, verbose = FALSE)
ari_mc <- round(mclust::adjustedRandIndex(mc_cmp$classification, cl_real), 3)

data.frame(
  Methode = c("K-means (K=3)", "CHA – Lien moyen (K=3)", "Mclust GMM (K=3)"),
  Donnees = c("V1, V2", "V1, V2", "V1, V2"),
  Forme = c("Sphérique", "Quelconque", "Elliptique"),
  K_auto = c("Non", "Coupure dendro", "BIC auto"),
  ARI = c(ari_km, ari_cah, ari_mc)
) |>
  setNames(c("Méthode", "Données", "Forme des clusters", "K automatique ?", "ARI
vs vérité terrain")) |>
  tt(caption = "Comparaison des méthodes sur les données Flynt & Dean (2016)") |>
  style_tt(j = 5, bold = TRUE)

```

Table 5

Méthode	Données	Forme des clusters	K automatique ?	ARI vs vérité terrain
K-means (K=3)	V1, V2	Sphérique	Non	0.798
CHA – Lien moyen (K=3)	V1, V2	Quelconque	Coupure dendro	0.534
Mclust GMM (K=3)	V1, V2	Elliptique	BIC auto	0.959

i Lecture du tableau

Les trois méthodes sont évaluées sur les mêmes variables continues (V_1, V_2) et la même vérité terrain, ce qui rend les ARI directement comparables. Mclust domine grâce à sa modélisation elliptique, tandis que la CHA souffre de ses fusions irréversibles (30% d'erreur).

17 Conclusion générale

17.1 Bilan du parcours : de R aux méthodes non supervisées

Ce document a constitué un voyage pédagogique en trois actes, tous ancrés dans l'unité d'enseignement ESD113 de Monsieur Karim KILANI.

Le **premier acte** nous a conduits des fondements de R et de Quarto jusqu'à la maîtrise du Tidyverse : manipulation de données, statistiques descriptives, visualisation, et pipe natif. Ces outils constituent la grammaire de base de tout travail d'analyse avec R.

Le **deuxième acte** a appliqué cette grammaire à un cas concret : les données de tirage du Keno. Chemin faisant, nous avons exploré le pivot, la modélisation probabiliste par la loi hypergéométrique, le calcul de l'espérance de gain et la simulation de Monte-Carlo. Ce dernier outil — répéter à grande échelle des tirages aléatoires pour estimer des distributions — est l'un des plus puissants de la boîte à outils statistique, et sa mise en œuvre en R s'avère remarquablement concise.

Le **troisième acte** a plongé dans les méthodes de clustering, en reproduisant et en enrichissant les résultats de l'article fondateur de Flynt & Dean (2016). Trois méthodes ont été passées au crible :

1. **K-means** : rapide, explicable, mais contraint par une hypothèse sphérique. Six variations d'implémentation ont montré qu'un même résultat statistique peut s'écrire de la ligne artisanale au graphique automatisé.
2. **La CHA** : précieuse pour l'exploration multi-échelle via le dendrogramme, mais pénalisée par ses fusions irréversibles (30% d'erreur).
3. **Mclust** : champion incontesté, avec **98,67% de réussite**, grâce à sa modélisation elliptique et à la sélection automatique du modèle via le BIC.

17.2 Ce que les données simulées nous ont appris

L'utilisation d'un fil conducteur unique — les données de Flynt & Dean (2016) — a mis en lumière une leçon centrale : le **groupe elliptique est le vrai révélateur** des hypothèses implicites de chaque algorithme. K-means (~7 % d'erreur sur ce groupe), CHA (30 % d'erreur globale), Mclust (1,33 % d'erreur) — ces chiffres ne décrivent pas trois niveaux de complexité informatique, mais **trois façons différentes de « voir » les données**.

17.3 La hiérarchie de performance (contexte-dépendante)

Pour ces données continues simulées, la hiérarchie de performance est claire :

```
$$\texttt{Mclust} \ ;>\; \texttt{K-means} \ ;>\; \texttt{CHA}$$$
```

Mclust>K-means>CHA

Situation	Méthode recommandée
Exploration rapide, N grand	K-means (nstart élevé)
Clusters gaussiens, K inconnu	Mclust (BIC automatique)
Lecture multi-échelle, exploration	CHA + dendrogramme

🔗 À retenir

Le clustering n'est pas une vérité : c'est une **hypothèse de travail** que l'analyste fait sur la structure des données. La vraie compétence du data scientist est de choisir — et de savoir justifier — pourquoi une méthode est plus adaptée que les autres au problème posé.

17.4 Applications concrètes du clustering dans la vie réelle

Le clustering n'est pas un exercice académique sur des données simulées : c'est l'un des outils les plus déployés dans les secteurs publics et privés.

Commerce et marketing — la segmentation client. Les enseignes de la grande distribution (Carrefour, Amazon, Fnac) appliquent quotidiennement le K-means ou les GMM à leurs bases de données d'achats. Chaque client se voit attribuer un segment qui pilote les campagnes e-mail, les recommandations produits et les programmes de fidélité. Spotify, Apple Music ou Netflix effectuent la même opération sur les comportements d'écoute ou de visionnage.

Santé et biologie — la médecine de précision. En oncologie, les chercheurs appliquent la CHA ou les GMM à des profils d'expression génique de tumeurs pour en identifier les sous-types moléculaires. C'est cette approche qui a conduit à distinguer plusieurs sous-types de cancer du sein (Luminal A, Luminal B, HER2-enrichi, Triple-négatif), chacun répondant différemment aux traitements.

Finance — la détection de fraude. Les départements de lutte anti-fraude des banques (BNP Paribas, Société Générale, Visa) utilisent des algorithmes de clustering non supervisé pour détecter les comportements de paiement anormaux. Un cluster de transactions nocturnes à l'étranger avec des montants inhabituels émerge automatiquement — sans qu'on ait besoin de définir à l'avance ce qu'est une fraude.

Défense et sécurité nationale — un enjeu croissant. L'analyse de signaux électromagnétiques (SIGINT) utilise des algorithmes de partitionnement pour regrouper automatiquement les émissions radio selon leur signature spectrale. La fusion de données ISR (Intelligence, Surveillance, Reconnaissance) applique le clustering spatio-temporel pour détecter des patterns comportementaux anormaux sur un théâtre d'opérations. En cybersécurité de défense, les CERT militaires (comme le CALID en France) utilisent la classification non supervisée de logs systèmes pour isoler automatiquement les comportements malveillants — notamment pour détecter les intrusions persistantes avancées (APT).

Image et vision par ordinateur. La compression d'image JPEG utilise le K-means pour réduire la palette de couleurs d'une image. La segmentation d'images satellitaires — qu'il s'agisse de cartographier des cultures agricoles ou de surveiller l'évolution du couvert forestier — repose sur des algorithmes de clustering appliqués aux pixels selon leur signature spectrale (visible, infrarouge, radar).

🔗 À retenir

Dans tous ces domaines, la leçon reste la même : le clustering révèle une structure que les données portent en elles mais que l'analyste n'avait pas définie *a priori*. Sa puissance réside précisément dans cette capacité à faire *émerger* de la connaissance à partir de l'observation brute, sans étiquette préalable.

17.5 Pour aller plus loin

- **Flynt et Dean** 2016 : la revue des packages R pour le clustering, article de référence de ce document.
- **Fraley et Raftery** 2002 : l'article fondateur de Mclust et des mélanges gaussiens.
- **Everitt et al.** 2011 : le livre de référence complet sur le clustering.
- **Wickham et al.** 2019 : l'écosystème tidyverse.

Références bibliographiques

Rapport de compilation

Méthode de mesure du temps

Lorsque vous cliquez sur **Render** dans RStudio, la compilation traverse trois phases dont seule la deuxième est mesurable depuis R :

Phase	Outil	Ce qui se passe	Mesurable ?
1 — Quarto/pan-doc	Quarto CLI	Lit le <code>.qmd</code> , prépare knitr, lance R	✗
2 — knitr (R)	knitr + R	Exécute tous les chunks. <code>proc.time()</code> lancé au <code>setup</code> , arrêté au <code>timer-final-typst</code> .	✓
3 — Rendu Typst	pandoc + typst	Convertit le <code>.md</code> en <code>.pdf</code> via le moteur Typst	✗

⚠ Pourquoi le total des chunks est inférieur au temps knitr global ?

La **somme des durées individuelles** mesure le temps CPU pur de chaque chunk. Le **chronomètre global knitr** est supérieur car il inclut aussi le rendu ggplot2, les appels système et les I/O disque *entre* les chunks. Les Phases 1 et 3 sont non mesurables depuis R.

i Disponibilité des données à la 2ème compilation

À la **première compilation**, `[KNITR_TOTAL]` n'est pas encore dans le fichier (écrit par `timer-final-typst` *après* les tableaux). **Dès la deuxième compilation**, toutes les valeurs apparaissent — le fichier `compilation_stats_typst.txt` contient les données de la session précédente.

Environnement système

Table 6

Paramètre	Valeur
Plateforme	unix
Système d'exploitation	Ubuntu 24.04.4 LTS
Version R	R version 4.5.3 (2026-03-11)
Processeur	AMD EPYC 7R13 Processor
Cœurs logiques	16
RAM totale (Go)	132.2
Début phase knitr	03/05/2026 07:22:25

Temps d'exécution des chunks R (Phase 2)

Table 7

Rang	Statut	Chunk R	Duree (sec)
1	LENT	fig-dendro-color	32.38
2	LENT	fig-dendro-circular	13.58
3	MOYEN	montecarlo-grande-echelle	1.84
4	MOYEN	fig-mclust-bic-fviz-v2	1.83
5	OK	fig-simul-donnees	0.71
6	OK	packages-clustering	0.65
7	OK	var6-factoextra	0.43
8	OK	fig-mclust-hull-v2	0.41
9	OK	fig-dendro-simple	0.40
10	OK	fig-mclust-classif-v2	0.39
11	OK	ggplot-hist	0.33
12	OK	fig-kmeans-eval	0.28
13	OK	plot-esperances	0.27
14	OK	fig-elbow	0.26
15	OK	palmares-fdj	0.25
16	OK	var2-for	0.23
17	OK	var4-purrr	0.23
18	OK	barchart-freq	0.22
19	OK	var1-manuelle	0.22
20	OK	var5-broom	0.22
21	OK	fig-mclust-ellipses-v2	0.22
22	OK	var3-sapply	0.21
23	OK	polar-chart	0.18

Rang	Statut	Chunk R	Duree (sec)
24	OK	freq-boules	0.17
25	OK	fig-mclust-uncertainty-v2	0.17
26	OK	mclust-fit	0.16
27	OK	tbl-comparaison-methodes	0.16
28	OK	load-data	0.15
29	OK	montecarlo-proba	0.15
30	OK	hist-base	0.07
31	OK	pivot-exemple	0.07
32	OK	gains-tableau-complet	0.07
33	OK	seq-entiers	0.06
34	OK	keno-long	0.06
35	OK	gains-calcul-long	0.06
36	OK	stats-desc	0.05
37	OK	tableau-gains	0.05
38	OK	gains-tableau-10	0.04
39	OK	esperances-grilles	0.04
40	OK	tbl-params-simulation	0.04
41	OK	calcul-cah	0.04
42	OK	tbl-sysinfo-typst	0.03
43	OK	tidyverse-select	0.02
44	OK	format-dates	0.02
45	OK	proba-gain	0.02
46	OK	esperance	0.02
47	OK	tbl-synthese-variations	0.02
48	OK	perf-kmeans	0.02
49	OK	tbl-perf-kmeans	0.02
50	OK	tbl-perf-mclust	0.02
51	OK	base-r	0.01
52	OK	filter-mpg	0.01
53	OK	pivot-keno	0.01
54	OK	gains-data	0.01
55	OK	montecarlo-simple	0.01
56	OK	montecarlo-replicate	0.01
57	OK	seq-cdm	0.00

Bilan global des phases

Table 8

Phase	Durée mesurée	Remarque
Phase 1 – Quarto / pandoc	Non mesurable	2-5 sec estimees
Phase 2a – Execution chunks R (somme)	57.70 sec	Mesure chunk par chunk
Phase 2b – Overhead knitr (I/O, rendus)	0.4 sec	Rendu ggplot2, I/O fichiers
Phase 2 – Total knitr mesure	58.1 sec	Seule valeur exacte
Phase 3 – Rendu Typst (pandoc)	Non mesurable	Generalement < 10 sec pour Typst

💡 Comment fonctionne le timer ?

Au lancement de chaque compilation, le chunk setup lit le fichier `compilation_stats_typst.txt` avant de l'écraser, et sauvegarde en mémoire le `[KNITR_TOTAL]` de la session précédente. Cette valeur est ainsi disponible **dès la première compilation** qui suit une compilation complète. Les tableaux affichent donc : la somme des chunks R de la compilation *en cours*, et le total knitr de la compilation *précédente* — ce qui est la seule approche fiable, puisque `timer-final-typst` écrit ce total *après* que tous les tableaux ont été générés.

Bibliographie

- [1] R Core Team, « R: A Language and Environment for Statistical Computing ». Vienna, Austria, 2024. [En ligne]. Disponible sur: <https://www.r-project.org/>
- [2] J. Allaire, C. Teague, C. Scheidegger, Y. Xie, et C. Dervieux, « Quarto ». 2024. doi: [10.5281/zenodo.5960048](https://doi.org/10.5281/zenodo.5960048).
- [3] H. Wickham *et al.*, « Welcome to the tidyverse », *Journal of Open Source Software*, vol. 4, n° 43, p. 1686, 2019, doi: [10.21105/joss.01686](https://doi.org/10.21105/joss.01686).
- [4] A. Flynt et N. Dean, « A Survey of Popular R Packages for Cluster Analysis », *Journal of Educational and Behavioral Statistics*, vol. 41, n° 2, p. 205-225, 2016, doi: [10.3102/1076998616631743](https://doi.org/10.3102/1076998616631743).
- [5] S. P. Lloyd, « Least Squares Quantization in PCM », *IEEE Transactions on Information Theory*, vol. 28, n° 2, p. 129-137, 1982, doi: [10.1109/TIT.1982.1056489](https://doi.org/10.1109/TIT.1982.1056489).
- [6] K. Pearson, « Contributions to the Mathematical Theory of Evolution », *Philosophical Transactions of the Royal Society of London A*, vol. 185, p. 71-110, 1894, doi: [10.1098/rsta.1894.0003](https://doi.org/10.1098/rsta.1894.0003).
- [7] B. S. Everitt, S. Landau, M. Leese, et D. Stahl, *Cluster Analysis*, 5^e éd. Chichester: John Wiley & Sons, 2011. doi: [10.1002/9780470977811](https://doi.org/10.1002/9780470977811).
- [8] J. H. Ward, « Hierarchical Grouping to Optimize an Objective Function », *Journal of the American Statistical Association*, vol. 58, n° 301, p. 236-244, 1963, doi: [10.1080/01621459.1963.10500845](https://doi.org/10.1080/01621459.1963.10500845).

- [9] J. MacQueen, « Some Methods for Classification and Analysis of Multivariate Observations », vol. 1, p. 281-297, 1967.
- [10] A. P. Dempster, N. M. Laird, et D. B. Rubin, « Maximum Likelihood from Incomplete Data via the EM Algorithm », *Journal of the Royal Statistical Society: Series B*, vol. 39, n° 1, p. 1-38, 1977, doi: [10.1111/j.2517-6161.1977.tb01600.x](https://doi.org/10.1111/j.2517-6161.1977.tb01600.x).
- [11] C. Fraley et A. E. Raftery, « Model-Based Clustering, Discriminant Analysis, and Density Estimation », *Journal of the American Statistical Association*, vol. 97, n° 458, p. 611-631, 2002, doi: [10.1198/016214502760047131](https://doi.org/10.1198/016214502760047131).
- [12] R. A. Johnson et D. W. Wichern, *Applied Multivariate Statistical Analysis*, 5^e éd. Upper Saddle River, NJ: Prentice Hall, 2002.
- [13] L. Scrucca, M. Fop, T. B. Murphy, et A. E. Raftery, « mclust 5: Clustering, Classification and Density Estimation Using Gaussian Finite Mixture Models », *The R Journal*, vol. 8, n° 1, p. 289-317, 2016, doi: [10.32614/RJ-2016-021](https://doi.org/10.32614/RJ-2016-021).
- [14] G. Schwarz, « Estimating the Dimension of a Model », *The Annals of Statistics*, vol. 6, n° 2, p. 461-464, 1978, doi: [10.1214/aos/1176344136](https://doi.org/10.1214/aos/1176344136).